J.B. INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC AUTONOMOUS)
Bhaskar Nagar, Moinabad Mandal, R.R. District, Hyderabad -500075

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

NODE JS/ REACT JS/ DJANGO
(R-25)

B. TECH II YEAR – II SEM

INDEX

S No.	Contents	Page No.
1	Vision, Mission & PEOs	iii
2	POs & PSOs	iv - v
3	Course Objectives & Outcomes	vi
4	Course Syllabus / List of Experiments	vii
5	1. Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.	1
	2. Make the above web application responsive web application using Bootstrap framework.	12
	3. Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.	25
	4. Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.	29
	5. Develop a java stand alone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.	32
	6. Create an xml for the bookstore. Validate the same using both DTD and XSD.	36
	7. Design a controller with servlet that provides the interaction with application developed in experiment 1 and the database created in experiment 5.	38
	8. Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)	39
	9. Create a custom server using http module and explore the other modules of Node JS like OS, path, event.	41
	10. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)	43
	11. For the above application create authorized end points using JWT (JSON Web Token).	59
	12. Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.	64
	13. Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js	73
	14. Create a TODO application in react with necessary components and deploy it into GitHub.	79
6	Viva Voce Questions	

1. Vision, Mission & PEOs

Vision

To be a centre of excellence in Computer Science and Engineering education and research, producing globally competent professionals with strong ethical values, innovative mind-set, and commitment to societal development.

Mission

- To provide high-quality education in computer science and engineering, fostering analytical thinking, technical expertise, and innovation.
- To promote research, industry interaction, and hands-on learning to address real-world challenges and emerging technologies.
- To nurture ethically responsible, socially conscious, and globally competent professionals committed to lifelong learning and societal development.

PEO's

Program Educational Objectives (PEOs)	
PEO1	Build successful careers , pursue higher education or embark on entrepreneurial ventures by applying strong foundations in computer science, engineering principles, and modern technologies.
PEO2	Design and develop effective computing solutions to real-world problems through critical thinking, research aptitude, and continuous learning.
PEO3	Demonstrate ethical conduct, leadership, teamwork, and a commitment to sustainable development and societal well-being.

2. POs & PSOs

Program Outcomes of CSE Department (POs):	
PO1	**Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	**Problem Analysis:** Identify**,** formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	**Design/Development of Solutions**: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	**Conduct Investigations of Complex Problems**: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modeling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	**Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modeling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	**The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	**Ethics**: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	**Individual and Collaborative Team work**: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	**Communication:** Communicate effectively and inclusively within the engineering Community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	**Project Management and Finance**: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in Multidisciplinary environments.
PO11	**Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes (PSOs):	
PSO1	Apply fundamental concepts of computer science to design, develop, and deploy reliable and efficient software systems using modern programming tools and methodologies.
PSO2	Analyze, model, and solve real-world problems using techniques in data science, artificial intelligence, and machine learning.
PSO3	Design and integrate computing systems with a focus on networking, cloud computing, cyber security, blockchain, and IoT for secure and scalable applications.

3. COURSE OBJECTIVES & OUTCOMES

S.No	COURSE OBJECTIVES
1	To implement the static web pages using HTML and do client-side validation using JavaScript.
2	To design and work with databases using Java.
3	To develop an end to end application using java full stack.
4	To introduce Node JS implementation for server-side programming.
5	To experiment with single page application development using React.

S. No	COURSE OUTCOMES- *The student will be able to:*
1	Build a custom website using HTML, CSS, Bootstrap, and basic JavaScript.
2	Demonstrate advanced JavaScript features and implement database connectivity using JDBC.
3	Develop server-side applications using Java technologies.
4	Implement server-side functionality using Node.js.
5	Design a Single Page Application (SPA) using React.

4. Course Syllabus/List of Experiments

S No.	Experiments
1	Build application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid a responsive web
2	Make the above web application responsive web application using Bootstrap framework.
3	Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.
4	Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.
5	Develop a java stand alone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.
6	Create an xml for the bookstore. Validate the same using both DTD and XSD.
7	Design a controller with servlet that provides the interaction with application developed in experiment 1 and the database created in experiment 5.
8	Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)
9	Create a custom server using http module and explore the other modules of Node JS like OS, path, event.
10	Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)
11	For the above application create authorized end points using JWT (JSON Web Token).
12	Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.
13	Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js
14	Create a TODO application in react with necessary components and deploy it into GitHub.

PROGRAMS

1. Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.

Responsive Web Design
Responsive web design is about creating web sites which automatically adjust themselves to look good on all devices, from small phones to large desktops. This is achieved using techniques like flexible grids, images, and CSS media queries to adjust content seamlessly. It avoids the need for separate mobile apps by ensuring usability and visual appeal across all viewports with one codebase.
.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="./style.css">
<title>Home - JB Online Book Store</title>
</head>
<body>
<div class="wrapper">
<div class="container">
<header>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tr>
<th width="20%"><img src="jblogo.png" alt="JB Online Book Store LOGO" width=""
height="100"/></th>
<th colspan=4>
<h1 style="color:white;">JB Online Book Store - Best Online Ebooks Store </h1>
</th>
</tr>
</table>
</header>
<nav>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tbody align="center" style="font-weight:bold;font-size:18px;"">
<tr>
<td width="20%"><hr><a href="index.html">Home</a><hr></td>
<td width="20%"><hr><a href="login.html">Login</a><hr></td>
<td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
<td width="20%"><hr><a href="catalog.html" >Catalog</a><hr></td>
<td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
</tr>
</tbody>
</table>
</nav>
</div>
<div class="container1">
<div class="sidebar1"></div>
<div class="container2">
<main>
<center>
<h2>Welcome to JB Online Book Store</h2>
```

```
<p> Discover the  <font size=5> joy of smart shopping at </font> JB Online Book Store! <font
size=5> get ready </font>
for <font size=5>this </font>.</br>special discount season <font
size=5>save </font>  up to <font size=5>90%</font> on all your purchases. Happy shopping!
        </br></p>
<br/><br/><br/><br/><br/>
<marquee behavior="alternate" scrolldelay="fast"  direction="right"> <img src="a.jpg"
    width="100" height="130" > </marquee>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2026 All rights reserved by JB Online Book Store
    </font></footer>
</div>
</body>
</html>
```

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="./style.css">
<title>Login - JB Online Book Store</title>
</head>
<body>
<div class="wrapper">
<div class="container">
<header>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tr>
<th width="20%"><img src="JBlogo.png" alt="JB Online Book Store LOGO" width="130"
height="100"/></th>
<th colspan=4>
<h1 style="color:white;">JB Online Book Store - Best Online Ebooks Store </h1>
</th>
</tr>
</table>
</header>
<nav>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tbody align="center" style="font-weight:bold;font-size:18px;"">
<tr>
<td width="20%"><hr><a href="index.html">Home</a><hr></td>
<td width="20%"><hr><a href="login.html">Login</a><hr></td>
<td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
<td width="20%"><hr><a href="catalog.html" >Catalog</a><hr></td>
<td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
</tr>
</tbody>
</table>
</nav>
```

```
</div>
<div class="container1">
<div class="sidebar1"></div>
<div class="container2">
<main>
<center><br>
<h3> Login Details</h3> <br/>
<form name="f1">
<table width="100%" align="center" >
<tr>
<td> User Name : </td>
<td> <input type="text" name="username"></td>
</tr>
<tr><td><br></td></tr>
<tr>
<td> Password : </td>
<td> <input type="password" name="password"></td>
</tr>
<tr><td><br></td></tr>
<tr><td></td>
<td><input type="submit" value="SUBMIT">
<input type="reset" value="RESET"></td>
</tr>
</table>
</form>
</center>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2026 All rights reserved by JB Online Book Store
</font></footer>
</div>
</body>
</html>
```

registration.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="./style.css">
<title>Registration - JB Online Book Store</title>
</head>
<body>
<div class="wrapper">
<div class="container">
<header>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tr>
<th width="20%"><img src="JBlogo.png" alt="JB Online Book Store LOGO" width="130"
height="100"/></th>
<th colspan=4>
```

```
<h1 style="color:white;">JB Online Book Store - Best Online Ebooks Store</h1>
</th>
</tr>
</table>
</header>
<nav>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tbody align="center" style="font-weight:bold;font-size:18px;"">
<tr>
<td width="20%"><hr><a href="index.html">Home</a><hr></td>
<td width="20%"><hr><a href="login.html">Login</a><hr></td>
<td width="20%"><hr><a href= "registration.html"> Registration</a><hr></td>
<td width="20%"><hr><a href="catalog.html" >Catalog</a><hr></td>
<td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
</tr>
</tbody>
</table>
</nav>
</div>
<div class="container1">
<div class="sidebar1"></div>
<div class="container2">
<main>
<center><br>
<h3>Registration Form </h3>
<br/>
<form name="f1">
<table cellpadding="1" align="center" >
<tr><td> Name:*</td>
<td><input type="text" name="username"></td></tr>
<tr><td>Password:*</td>
<td><input type="password" name="password"></td></tr>
<tr><td>Email ID:*</td>
<td><input type="text" name="email"></td></tr>
<tr><td>Phone Number:*</td>
<td><input type="text" name="phno"></td></tr>
<tr><td valign="top">Gender:*</td>
<td><input type="radio" name="radio" value="1">Male   
<input type="radio" name="radio" value="2">Female</td></tr>
<tr> <td valign="top">Language Known:*</td>
<td> <input type="checkbox" name="checkbox"
value="English">English<br/>
<input type="checkbox" name="checkbox" value="Telugu">Telugu<br>
<input type="checkbox" name="checkbox" value="Hindi">Hindi<br>
<input type="checkbox" name="checkbox" value="Tamil">Tamil
</td></tr>
<tr> <td valign="top">Address:*</td>
<td><textarea name="address"></textarea></td>
<tr><td></td><td><input type="submit" value="submit"
hspace="10">
<input type="reset" value="reset"></td></tr>
<tr> <td colspan=2 >*<font color="#FF0000">fields are
mandatory</font>
```

```
</td>
</tr>
</table>
</form>
</center>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2026 All rights reserved by JB Online Book Store
 </font></footer>
</div>
</body>
</html>
```

cart.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="./style.css">
<title>Cart - JB Online Book Store</title>
</head>
<body>
<div class="wrapper">
<div class="container">
<header>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tr>
<th width="20%"><img src="JBlogo.png" alt="JB Online Book Store LOGO" width="130"
height="100"/></th>
<th colspan=4>
<h1 style="color:white;">JB Online Book Store - Best Online Ebooks Store</h1>
</th>
</tr>
</table>
</header>
<nav>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tbody align="center" style="font-weight:bold;font-size:18px;"">
<tr>
<td width="20%"><hr><a href="index.html">Home</a><hr></td>
<td width="20%"><hr><a href="login.html">Login</a><hr></td>
<td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
<td width="20%"><hr><a href="catalog.html" >Catalog</a><hr></td>
<td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
</tr>
</tbody>
</table>
</nav>
</div>
<div class="container1">
<div class="sidebar1"></div>
```

```
<div class="container2">
<main>
<center>
<h3>Cart</h3>
<table width="100%" align="center" >
<tbody>
<tr>
<th width="40%"><hr>BookName<hr></th>
<th width="20%"><hr>Price<hr></th>
<th width="20%"><hr>Quantity<hr></th>
<th width="20%"><hr>Amount<hr></th> </tr>
</tbody>
<tbody align=center>
<tr> <td>Java Programming </td>
<td>Rs. 2300/-</td>
<td>2</td>
<td>Rs. 4600/-</td></tr>
<tr><td>Web Technologies</td>
<td>Rs. 3000/-</td>
<td>1</td>
<td>Rs. 3000/-</td></tr>
<tr><td></td>
<td><hr><font color="#996600">Total Amount:</font><hr></td>
<td><hr>3<hr></td>
<td><hr>Rs. 7600/-<hr></td> </tr>
</tbody>
</table>
</center>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2026 All rights reserved by JB Online Book Store
</font></footer>
</div>
</body>
</html>
```

Catalog.html

```
<!DOCTYPE html>
<html>
<head>
<script src="script.js"> </script>
<link rel="stylesheet" href="./style.css">
<title>Catalog - JB Online Book Store</title>
</head>

<body>
<div class="wrapper">
<div class="container">
<header>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tr>
```

```
<th width="20%"><img src="JBlogo.png" alt="JB Online Book Store LOGO" width="130"
height="100"/></th>
<th colspan=4>
<h1 style="color:white;">JB Online Book Store - Best Online Ebooks Store </h1>
</th>
</tr>
</table>
</header>
<nav>
<table width="100%" align="center" cellpadding="0" cellspacing="2">
<tbody align="center" style="font-weight:bold;font-size:18px;"">
<tr>
<td width="20%"><hr><a href="index.html">Home</a><hr></td>
<td width="20%"><hr><a href="login.html">Login</a><hr></td>
<td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
<td width="20%"><hr><a href="catalog.html" >Catalog</a><hr></td>
<td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
</tr>
</tbody>
</table>
</nav>
</div>
<div class="container1">
<div class="sidebar1"></div>
<div class="container2">
<main>

<center>
    <div class="catalog">
      <h2>Books Catalog</h2>
    <div class="product">
      <h3>Java Programming</h3>
      <p>₹2300</p>
      <button type="button" onclick="addToCart('Java Programming', '₹2300')">
        Add to Cart
      </button>
    </div>

    <div class="product">
      <h3> Web Technologies</h3>
      <p>₹3000</p>
      <button type="button" onclick="addToCart('Web Technologies', '₹3000')">
        Add to Cart
      </button>
    </div>
</main>
</div>
<div class="sidebar2"></div>
</div>
<footer><font color="white">(C) 2026 All rights reserved by JB Online Book Store
</font></footer>
</div>
</body>
</html>
```

```
    </div>

    <br>

    <br>
</body>
</html>
```

style.css

```
body{
  font-family: monospace;
}

main {
  background-color: #efefef;
  color: #330000;
  margin-left: 10px;
  height: 60vh;
}

 header, footer {
  background-color: #000d57;
  color: #fff;
  padding: 1rem;
  height: 50px;
}

header, nav{
  margin-bottom: 10px;
  flex-basis: 50%;
}

footer{
  margin-top: 10px;
}
nav {
  background-color: #fff;
  color: #000;
  padding: 1rem;
  height: 20px;
}

.sidebar1, .sidebar2 {
  flex-basis: 10%;
  background-color: #fff;
  color: #000;
}

.sidebar2{
  margin-left: 10px;
}

.container1{
```

```
  display: flex;
}

.container2 {
  display: flex;
  flex-direction: column;
  flex: 1;
}

header, nav, main, .sidebar1, .sidebar2, footer{
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 10px;
}

.wrapper {
  display: flex;
  flex-direction: column;
  font-weight: 600;
}
```

Output Screens:

JB Online Book Store - Best Online Ebooks Store
Home
Login
Registration
Catalog
Cart
Login Details
User Name :
Password :
SUBMIT
RESET
(C) 2026 All rights reserved by JB Online Book Store

JB Online Book Store - Best Online Ebooks Store
Home
Login
Registration
Catalog
Cart
Registration Form
Name:*
Password:*
Email ID:*
Phone Number:*
Gender:*
Male
Female
Language Known:*
English
Telugu
Hindi
Tamil
Address:*
submit
reset
*fields are mandatory
(C) 2026 All rights reserved by JB Online Book Store

JB Online Book Store - Best Online Ebooks Store
Home
Login
Registration
Catalog
Cart
Books Catalog
Java Programming
₹2300
Add to Cart
Web Technologies
₹3000
Add to Cart
(C) 2026 All rights reserved by JB Online Book Store

JB Online Book Store - Best Online Ebooks Store

Home | Login | Registration | Catalog | Cart

Cart

BookName	Price	Quantity	Amount
Java Programming	Rs. 2300/-	2	Rs. 4600/-
Web Technologies	Rs. 3000/-	1	Rs. 3000/-
	Total Amount:	3	Rs. 7600/-

(C) 2026 All rights reserved by JB Online Book Store

2. **Make the above web application responsive web application using Bootstrap framework**

Registration.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <link rel="stylesheet" href="./style.css">
   <title>Registration - JBS</title>
  <script language="javascript">
   function validate() {
      // username validation
      var uname = f1.username.value;
      if (uname.length<=0)
      {
         alert("Please Enter UserName");
         f1.username.focus();
         return false;
      }
      if (uname.length < 8)
      {
         alert("Please enter UserName not less than 8");
         f1.username.focus();
         return false;
      }
      //password validation
      var pwd = f1.password.value;
      if (pwd.length<=0)
      {
         alert("Please Enter password");
         f1.password.focus();
         return false;
      }
      if (pwd.length < 6)
      {
         alert("Please enter Password not less than 6");
         f1.password.focus();
         return false;
      }
      // email validation
      var email = f1.email.value;
      if (email.length<=0)
      {
         alert("Please Enter email");
         f1.email.focus();
         return false;
      }
      else {
         let eflag=false;
         for(i=0;i<email.length;i++) {
            if(email.charAt(i)=="@")
               {
                  eflag=true;
```

```
            }
        }
        if(!(eflag))
        {
            alert("Please enter a valid Email ID");
            f1.email.focus();
            return false;
        }
    }
    // phone number validation
    var phno = f1.phno.value;
    if (phno.length<=0)
    {
        alert("Please Enter Phone Number");
        f1.phno.focus();
        return false;
    }
    if (isNaN(phno))
    {
        alert("Please Enter Valid Phone Number");
        f1.phno.focus();
        return false;
    }
    if (phno.length != 10)
    {
        alert("Please Enter Valid Phone Number");
        f1.phno.focus();
        return false;
    }
    // gender validation
    let flag=false;
    for(i=0;i<f1.gen.length;i++)
        if(f1.gen[i].checked)
            flag=true;
    if(!(flag))
    {
    alert("Please choose a Gender");
    return false;
    }
    // Language validation
    flag=false;
    for(i=0;i<f1.lang.length;i++)
        if(f1.lang[i].checked)
            flag=true;
    if(!(flag))
    {
    alert("Please select at least one of the Language options.");
    return false;
    }
    // address validation
    var addr = f1.address.value;
    if (addr.length<=0)
    {
        alert("Please Enter address");
```

```
            f1.address.focus();
            return false;
        }
        // to display Success message
        alert("Registration Successful");
    }
    </script>

  </head>
  <body>
    <div class="wrapper">
      <div class="container">
        <header>
          <table width="100%" align="center" cellpadding="0" cellspacing="2">
          <tr>
          <th width="20%"><img src="jblogo.jpg" alt="JBS LOGO"  width="130"
     height="100"/></th>
          <th colspan=4>
          <h1 style="color:white;">JBS - WORLD BEST ONLINE EBOOKS WEBSITE</h1>
          </th>
          </tr>
          </table>
        </header>
        <nav>
          <table width="100%" align="center" cellpadding="0" cellspacing="2">
          <tbody align="center"  style="font-weight:bold;font-size:18px;"">
          <tr>
          <td width="20%"><hr><a href="index.html">Home</a><hr></td>
          <td width="20%"><hr><a href="login.html">Login</a><hr></td>
          <td width="20%"><hr><a href="registration.html">Registration</a><hr></td>
          <td width="20%"><hr><a href="cart.html" >Cart</a><hr></td>
          </tr>
          </tbody>
          </table>
        </nav>
      </div>
      <div class="container1">
        <div class="sidebar1"></div>
        <div class="container2">
          <main>
            <center><br>
              <h3>Registration Form </h3>
              <br/>
              <form name="f1">
              <table cellpadding="1" align="center" >
              <tr><td> Name:*</td>
              <td><input type="text" name="username"></td></tr>
              <tr><td>Password:*</td>
              <td><input type="password" name="password"></td></tr>
              <tr><td>Email ID:*</td>
              <td><input type="text" name="email"></td></tr>
              <tr><td>Phone Number:*</td>
              <td><input type="text" name="phno"></td></tr>
              <tr><td valign="top">Gender:*</td>
```

```
                <td><input type="radio" name="radio" value="1">Male   
                <input type="radio" name="radio" value="2">Female</td></tr>
                <tr> <td valign="top">Language Known:*</td>
                <td> <input type="checkbox" name="checkbox" value="English">English<br/>
                <input type="checkbox" name="checkbox" value="Telugu">Telugu<br>
                <input type="checkbox" name="checkbox" value="Hindi">Hindi<br>
                <input type="checkbox" name="checkbox" value="Tamil">Tamil
                </td></tr>
                <tr> <td valign="top">Address:*</td>
                <td><textarea name="address"></textarea></td>
                <tr><td></td>
            <td><input type="submit" value="submit" hspace="10"  onclick="validate()">
                <input type="reset" value="reset"></td></tr>
                <tr> <td colspan=2 >*<font color="#FF0000">fields are mandatory</font>
                </td>
                </tr>
                </table>
                </form>
              </center>
            </main>
          </div>
          <div class="sidebar2"></div>
        </div>
        <footer><font color="white">(C) 2024 All rights reserved by JBS ebooks</font></footer>
      </div>
    </body>
    </html>
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Home - JB Book Store</title>
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/css/bootstrap.min.css" rel="stylesheet">
<style>
body {
background-color: #f8f9fa;
}
.header-img {
height: 100px;
}
.custom-header {
background-color: #ffffff;
color: white;
padding: 20px 0;
}
.custom-nav a {
color: white;
text-decoration: none;
```

```
padding: 10px 20px;
display: inline-block;
}
.custom-nav a:hover {
border-radius: 5px;
}
nav{
background-color:#0b6620;
}
</style>
<link
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container-fluid">
<div class="container-fluid">
<!-- Header Section -->
<header class="custom-header text-center">
<div class="container d-flex align-items-center justify-contentbetween">
<img src="jblogo.jpg" alt="JB Logo" class="header-img">
<h1 class="mb-0">JB Online Book Store - Best Online Ebooks Store</h1>
</div>
</header>
<!-- Navigation Section -->
<nav class="custom-nav text-center">
<a href="index.html">Home</a>
<a href="login.html">Login</a>
<a href="registration.html">Register</a>
<a href="cart.html">Cart</a>
</nav>
<!-- Main Content -->
<main class="mt-4">
<div class="container text-center">
<h2>Welcome to JB Online Book Store </h2>
<img src="jblogo.jpg" alt="" width="10%" height="10%">
<p>Your one-stop destination for online eBooks.</p>
<button class="btn btn-primary">Explore Now</button>
</div>
</main>
<!-- Footer Section -->
<footer class="bg-dark text-white text-center py-3 mt-4">
<p>&copy; 2026 JB Online Book Store. All rights reserved.</p>
</footer>
</div>
<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</div>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Login - JB Book Store</title>
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/css/bootstrap.min.css" rel="stylesheet">
<style>
body {
background-color: #f8f9fa;
}
.header-img {
height: 100px;
}
.custom-header {
background-color: #ffffff;
color: white;
padding: 20px 0;
}
.custom-nav a {
color: white;
text-decoration: none;
padding: 10px 20px;
display: inline-block;
}
.custom-nav a:hover {
background-color: #495057;
border-radius: 5px;
}
</style>
<link
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container-fluid">
<div class="container-fluid">
<!-- Header Section -->
<header class="custom-header text-center">
<div class="container d-flex align-items-center justify-contentbetween">
<img src="jblogo.jpg" alt="JB LOGO" class="header-img">
<h1 class="mb-0">JB Online Book Store - Best Online Ebooks Store</h1>
</div>
</header>
<!-- Navigation Section -->
<nav class="bg-dark custom-nav text-center">
<a href="index.html">Home</a>
<a href="login.html">Login</a>
<a href="registration.html">Register</a>
<a href="cart.html">Cart</a>
</nav>
```

```
<!-- Login Form Section -->
<main class="mt-4">
<div class="container">
<h2 class="text-center">Login</h2>
<form class="mx-auto mt-3" style="max-width: 400px;">
<div class="mb-3">
<label for="username" class="form-label">Username</label>
<input type="text" class="form-control" id="username"
placeholder="Enter your username">
</div>
<div class="mb-3">
<label for="password" class="form-label">Password</label>
<input type="password" class="form-control" id="password"
placeholder="Enter your password">
</div>
<div class="mb-3 form-check">
<input type="checkbox" class="form-check-input"
id="rememberMe">
<label class="form-check-label" for="rememberMe">Remember
Me</label>
</div>
<button type="submit" class="btn btn-primary w-
100">Login</button>
</form>
</div>
</main>
<!-- Footer Section -->
<footer class="bg-dark text-white text-center py-3 mt-4">
<p>&copy; 2026 JB Online Book Store. All rights reserved.</p>
</footer>
</div>
<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</div>
</body>
</html>
```

Registration.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Registration Page</title>
<!-- Bootstrap CSS -->
<link
href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/css/bootstrap.min.cs
s" rel="stylesheet">
<link rel="stylesheet" href="stylecss">
<style>
body {
background-color: #f8f9fa;
```

```
}
.header-img {
height: 100px;
}
.custom-header {
background-color: #ffffff;
color: white;
padding: 20px 0;
}
.custom-nav a {
color: white;
text-decoration: none;
padding: 10px 20px;
display: inline-block;
}
.custom-nav a:hover {
background-color: #495057;
border-radius: 5px;
}
</style>
<link
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container-fluid">
<!-- Header Section -->
<header class="custom-header text-center">
<div class="container d-flex align-items-center justify-content-between">
<img src="jblogo.jpg" alt="jb logo" class="header-img">
<h1 class="mb-0">JB Online Book Store - Best Online Ebooks Store</h1>
</div>
</header>
<!-- Navigation Section -->
<nav class="bg-dark custom-nav text-center">
<a href="index.html">Home</a>
<a href="login.html">Login</a>
<a href="registration.html">Register</a>
<a href="cart.html">Cart</a>
</nav>
<!-- Main Content -->
<main class="container my-5">
<h1 class="text-center mb-4">Registration</h1>
<form class="row g-3">
<div class="col-md-6">
<label for="firstName" class="form-label">First Name</label>
<input type="text" class="form-control" id="firstName"
placeholder="Enter your first name">
</div>
<div class="col-md-6">
<label for="lastName" class="form-label">Last Name</label>
<input type="text" class="form-control" id="lastName"
placeholder="Enter your last name">
</div>
```

```
<div class="col-md-12">
<label for="email" class="form-label">Email</label>
<input type="email" class="form-control" id="email"
placeholder="Enter your email">
</div>
<div class="col-md-12">
<label for="password" class="form-label">Password</label>
<input type="password" class="form-control" id="password"
placeholder="Enter your password">
</div>
<div class="col-md-12">
<label for="confirmPassword" class="form-label">Confirm
Password</label>
<input type="password" class="form-control" id="confirmPassword"
placeholder="Confirm your password">
</div>
<div class="col-12">
<button type="submit" class="btn btn-primary w-
100">Register</button>
</div>
</form>
</main>
<!-- Footer -->
<footer class="bg-dark text-white text-center py-3">
<p class="mb-0">&copy; 2026 JB Online Book Store. All rights reserved.</p>
</footer>
<!-- Bootstrap JS -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/js/bootstrap.bundle.m
in.js"></script>
</div>
</body>
</html>
```

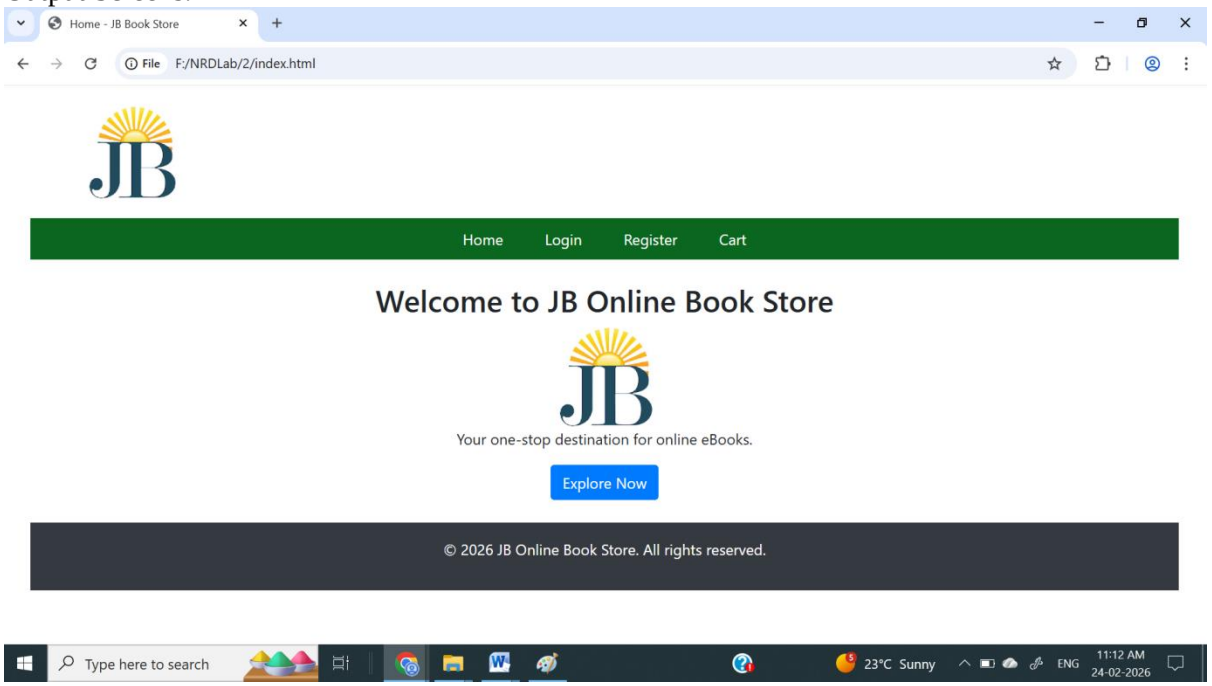
Cart.html

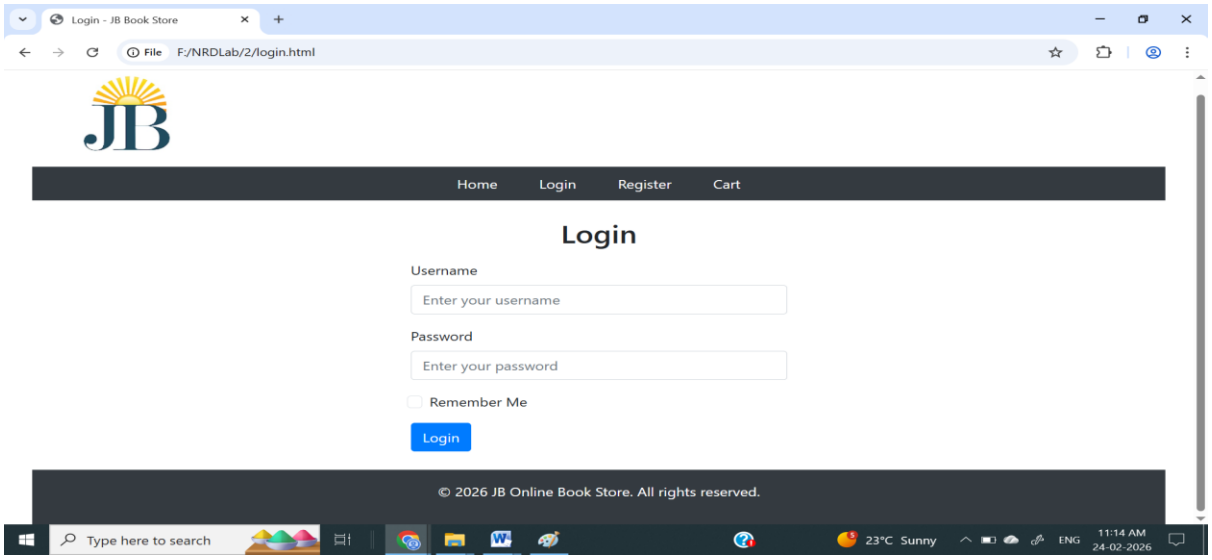
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Cart Page</title>
<!-- Bootstrap CSS -->
<link
href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/css/bootstrap.min.cs
s" rel="stylesheet">
<link rel="stylesheet" href="stylecss">
<style>
body {
background-color: #f8f9fa;
}
.header-img {
height: 100px;
}
```

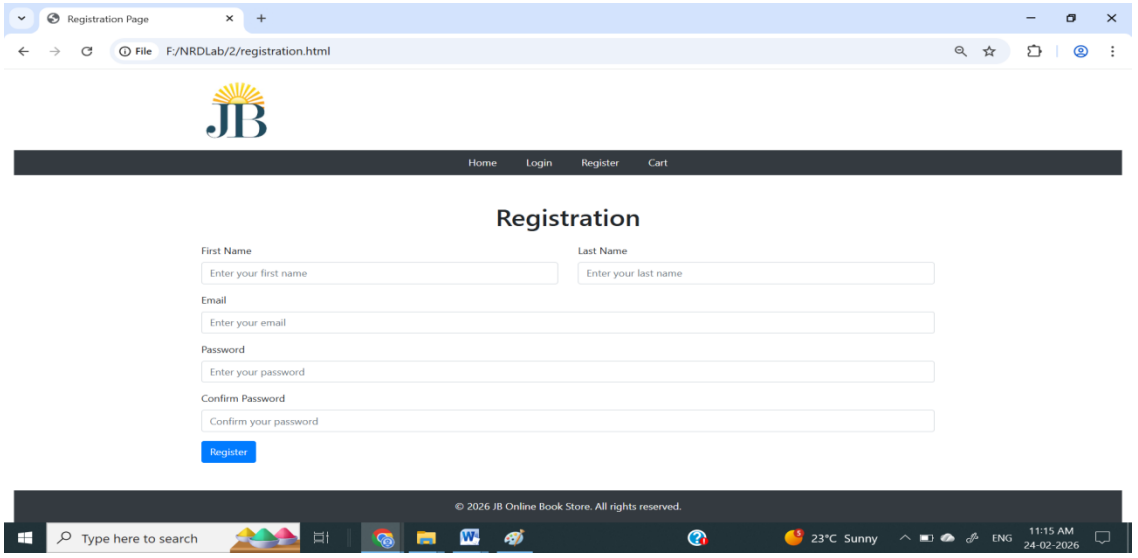
```
.custom-header {
background-color: #ffffff;
color: white;
padding: 20px 0;
}
.custom-nav a {
color: white;
text-decoration: none;
padding: 10px 20px;
display: inline-block;
}
.custom-nav a:hover {
background-color: #495057;
border-radius: 5px;
}
</style>
<link
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container-fluid">
<!-- Header Section -->
<header class="custom-header text-center">
<div class="container d-flex align-items-center justify-content-between">
<img src="jblogo.jpg" alt="JB LOGO" class="header-img">
<h1 class="mb-0">JB Online Book Store - Best Online Ebooks Store</h1>
</div>
</header>
<!-- Navigation Section -->
<nav class="bg-dark custom-nav text-center">
<a href="index.html">Home</a>
<a href="login.html">Login</a>
<a href="registration.html">Register</a>
<a href="cart.html">Cart</a>
</nav>
<!-- Main Content -->
<main class="container my-5">
<h1 class="text-center mb-4">Your Cart</h1>
<div class="table-responsive">
<table class="table table-bordered">
<thead class="table-dark">
<tr>
<th>Item</th>
<th>Description</th>
<th>Price</th>
<th>Quantity</th>
<th>Total</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
<tr>
<td> 1 </td>
```

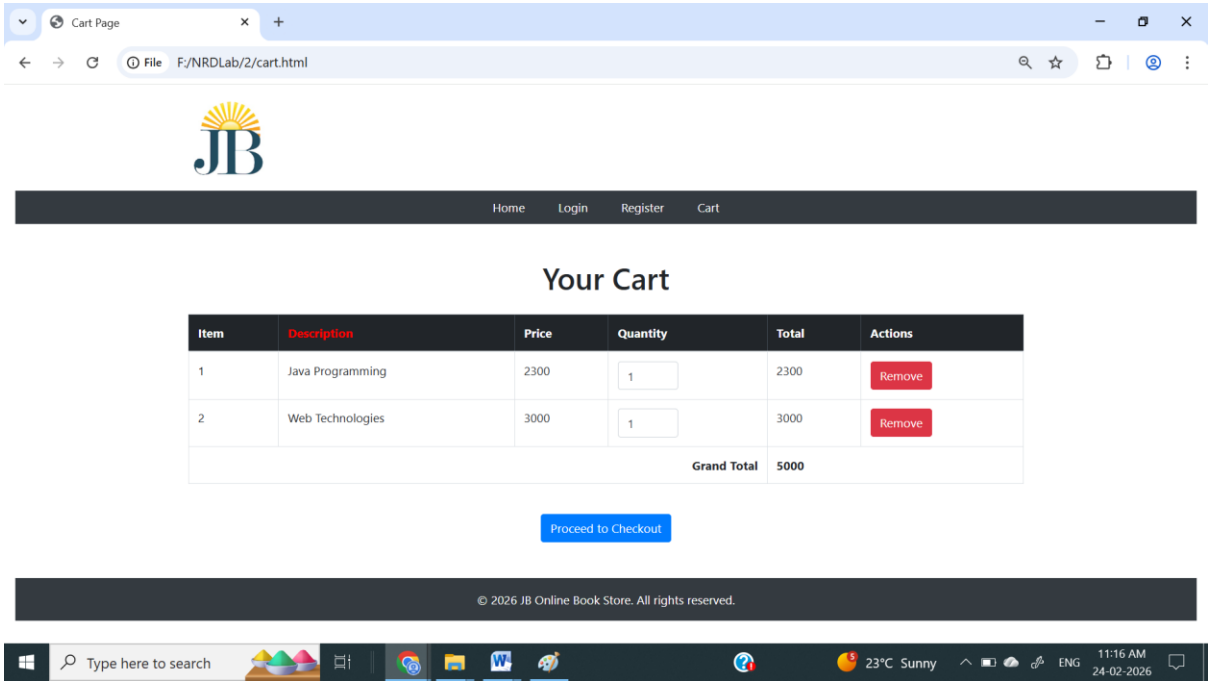
```
<td>Java Programming</td>
<td> 2300</td>
<td><input type="number" value="1" class="form-control"
style="width: 80px;"></td>
<td>2300</td>
<td><button class="btn btn-danger btnsm">
Remove</button></td>
</tr>
<tr>
<td>2 </td>
<td>Web Technologies</td>
<td>3000</td>
<td><input type="number" value="1" class="form-control"
style="width: 80px;"></td>
<td>3000</td>
<td><button class="btn btn-danger btnsm">
Remove</button></td>
</tr>
</tbody>
<tfoot>
<tr>
<th colspan="4" class="text-end">Grand Total</th>
<th colspan="2">5000</th>
</tr>
</tfoot>
</table>
</div>
<div class="text-center mt-4">
<button class="btn btn-primary">Proceed to Checkout</button>
</div>
</main>
<!-- Footer -->
<footer class="bg-dark text-white text-center py-3">
<p class="mb-0">&copy; 2026 JB Online Book Store. All rights reserved.</p>
</footer>
<!-- Bootstrap JS -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/js/bootstrap.bundle.m
in.js"></script>
</div>
</body>
</html>
```

Output Screens:








Cart Page
File F:/NRDLab/2/cart.html
JB
Home Login Register Cart
Your Cart
Item Description Price Quantity Total Actions
1 Java Programming 2300 1 2300 Remove
2 Web Technologies 3000 1 3000 Remove
Grand Total 5000
Proceed to Checkout
© 2026 JB Online Book Store. All rights reserved.
Type here to search
23°C Sunny
ENG
11:16 AM
24-02-2026

3. Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.

Client - Side validation of Registration Page

registrationJS.html

```
<html>
<head>
   <title> Welcome to NNRG e-Book's website</title>
   <script language="javascript">
   function validate() {
      // username validation
      var uname = f1.username.value;
      if (uname.length<=0)
      {
         alert("Please Enter UserName");
         f1.username.focus();
         return false;
      }
      if (uname.length < 8)
      {
         alert("Please enter UserName not less than 8");
         f1.username.focus();
         return false;
      }
      //password validation
      var pwd = f1.password.value;
      if (pwd.length<=0)
      {
         alert("Please Enter password");
         f1.password.focus();
         return false;
      }
      if (pwd.length < 6)
      {
         alert("Please enter Password not less than 6");
         f1.password.focus();
         return false;
      }
      // email validation
      var email = f1.email.value;
      if (email.length<=0)
      {
         alert("Please Enter email");
         f1.email.focus();
         return false;
      }
      else {
         let eflag=false;
         for(i=0;i<email.length;i++) {
            if(email.charAt(i)=="@")
               {
                  eflag=true;
```

```
            }
        }
        if(!(eflag))
        {
            alert("Please enter a valid Email ID");
            f1.email.focus();
            return false;
        }
    }
    // phone number validation
    var phno = f1.phno.value;
    if (phno.length<=0)
    {
        alert("Please Enter Phone Number");
        f1.phno.focus();
        return false;
    }
    if (isNaN(phno))
    {
        alert("Please Enter Valid Phone Number");
        f1.phno.focus();
        return false;
    }
    if (phno.length != 10)
    {
        alert("Please Enter Valid Phone Number");
        f1.phno.focus();
        return false;
    }
    // gender validation
    let flag=false;
    for(i=0;i<f1.gen.length;i++)
        if(f1.gen[i].checked)
            flag=true;
    if(!(flag))
    {
    alert("Please choose a Gender");
    return false;
    }
    // Language validation
    flag=false;
    for(i=0;i<f1.lang.length;i++)
        if(f1.lang[i].checked)
            flag=true;
    if(!(flag))
    {
    alert("Please select at least one of the Language options.");
    return false;
    }
    // address validation
    var addr = f1.address.value;
    if (addr.length<=0)
    {
        alert("Please Enter address");
```

```
        f1.address.focus();
        return false;
      }
      // to display Success message
      alert("Registration Successful");
    }
    </script>
  </head>
  <body>
  <center><br>
    <h3>Registration Form </h3>
    <br/>
    <form name="f1">
    <table cellpadding="1" align="center" >
    <tr><td> User Name:*</td>
    <td><input type="text" name="username"></td></tr>
    <tr><td>Password:*</td>
    <td><input type="password" name="password"></td></tr>
    <tr><td>Email ID:*</td>
    <td><input type="text" name="email"></td></tr>
    <tr><td>Phone Number:*</td>
    <td><input type="text" name="phno"></td></tr>
    <tr><td valign="top">Gender:*</td>
    <td><input type="radio" name="gen" value="Male">Male   
    <input type="radio" name="gen" value="Female">Female</td></tr>
    <tr> <td valign="top">Language Known:*</td>
    <td> <input type="checkbox" name="lang" value="English">English<br/>
    <input type="checkbox" name="lang" value="Telugu">Telugu<br>
    <input type="checkbox" name="lang" value="Hindi">Hindi<br>
    <input type="checkbox" name="lang" value="Tamil">Tamil
    </td></tr>
    <tr> <td valign="top">Address:*</td>
    <td><textarea name="address"></textarea></td>
    <tr><td></td><td><input type="button" value="SUBMIT" hspace="10"
      onclick="validate()">
    <input type="reset" value="RESET"></td></tr>
    <tr> <td colspan=2 >*<font color="#FF0000">fields are mandatory</font>
    </td>
    </tr>
    </table>
    </form>
  </center>
  </body>
  </html>
```

i. **Add onsubmit="return validateLogin()" to your <form> tag of Login.html**
   Syntax
   <form name="f1" onsubmit="return validateLogin()">

ii. **Add a <script> tag just before your closing </body> tag and paste the function inside it:**

```
<script>
 function validateLogin() {
const username = document.f1.username.value.trim();
const password = document.f1.password.value.trim();
```

```
  // Username validation
  if (username === "") {
    alert("Username cannot be empty.");
    document.f1.username.focus();
    return false;
  }

  if (username.length < 4) {
    alert("Username must be at least 4 characters long.");
    document.f1.username.focus();
    return false;
  }

  // Password validation
  if (password === "") {
    alert("Password cannot be empty.");
    document.f1.password.focus();
    return false;
  }
  if (password.length < 6) {
    alert("Password must be at least 6 characters long.");
    document.f1.password.focus();
    return false;
  }
  const hasUpperCase = /[A-Z]/.test(password);
  const hasLowerCase = /[a-z]/.test(password);
  const hasNumber   = /[0-9]/.test(password);

  if (!hasUpperCase || !hasLowerCase || !hasNumber) {
    alert("Password must contain at least one uppercase letter, one lowercase letter, and one
number.");
    document.f1.password.focus();
    return false;
  }

  alert("Login successful! Welcome, " + username + "!");
  return true;
}
  </script>
```

4. **Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.**

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="styles.css">
  <title>Weather Graph</title>
</head>
<body>
  <div class="container">
    <h1>Weather Graph</h1>
    <canvas id="weatherGraph" width="400" height="200"></canvas>
  </div>
  <!-- Include third-party libraries from their CDNs -->
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <!-- Link to your local script file -->
  <script src="script.js"></script>
</body>
</html>
```

script.js

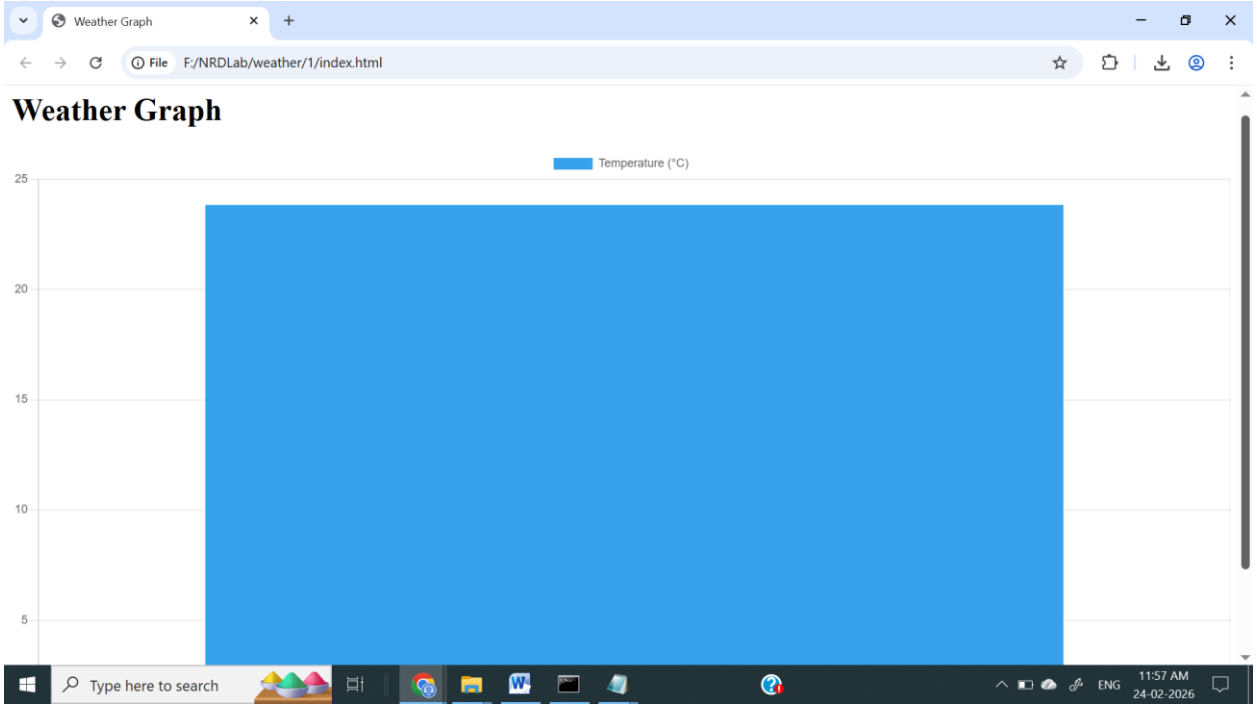
```
document.addEventListener('DOMContentLoaded', () => {
  const apiKey = 'e4487e40ea5b6d108d6382f7bd465cd6';
  const city = 'Hyderabad';
  const apiUrl =
`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=m
etric`;

  // Fetch weather data from API
  const fetchData = async () => {
    try {
      const response = await axios.get(apiUrl);
      const weatherData = response.data;
      updateGraph(weatherData.main.temp);
    } catch (error) {
      console.error('Error fetching weather data:', error.message);
    }
  };

  // Update the chart with fetched temperature
  const updateGraph = (temperature) => {
    const ctx = document.getElementById('weatherGraph').getContext('2d');
```

```
        // Initialize Chart.js
        new Chart(ctx, {
            type: 'bar',
            data: {
                labels: ['Temperature'],
                datasets: [{
                    label: 'Temperature (°C)',
                    data: [temperature],
                    backgroundColor: ['#36A2EB'],
                }],
            },
            options: {
                scales: {
                    y: {
                        beginAtZero: true, // Fixed spacing error here
                    },
                },
            },
        });
    };

    // Run on load
    fetchData();
});
```

style.css

```
/* Styles the entire body of the HTML document */
body {
    font-family: 'Arial', sans-serif; /* Sets the primary font */
    margin: 0;                        /* Removes default browser margin */
    padding: 0;                       /* Removes default browser padding */
    background-color: #f4f4f4;        /* Sets a light gray background color */
}

/* Styles elements with the class 'container' (e.g., <div class="container">) */
.container {
    max-width: 600px;                 /* Limits the maximum width of the container */
    margin: 50px auto;                /* Centers the container horizontally and adds top/bottom
margin */
    background-color: #fff;           /* Sets a white background */
    padding: 20px;                    /* Adds internal spacing around the content */
    border-radius: 8px;               /* Rounds the corners of the container */
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); /* Adds a subtle shadow for depth */
}

/* Styles all <h1> heading elements */
h1 {
    text-align: center;               /* Centers the text within the heading */
}

/* Styles all <canvas> elements (typically used for graphics) */
```

```
canvas {
   display: block;              /* Ensures the canvas takes up its own block of space */
   margin: 20px auto;              /* Centers the canvas horizontally and adds top/bottom
margin */
}
```



5. **Develop a java stand alone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.**

Step1: create the following table in Mysql

```
CREATE TABLE IF NOT EXISTS student (
        s_id            INT     PRIMARY KEY,
        s_name          VARCHAR(255),
        s_address       VARCHAR(255)
        );
```

Step 2: Inserting data into table through Java Program

<u>InsertData.java</u>

```
import java.sql.*;
import java.util.Scanner;

public class InsertData {
   public static void main(String[] args) {
      try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/mydb", "root", "");
         Statements = con.createStatement()) {

         Scanner sc = new Scanner(System.in);
         System.out.println("Inserting Data into student table:");
         System.out.println("_____________________________________");

         System.out.print("Enter student id: ");
         int sid = sc.nextInt();
         System.out.print("Enter student name: ");
         String sname = sc.next();
         System.out.print("Enter student address: ");
         String saddr = sc.next();

         String insertQuery = "INSERT INTO student VALUES(" + sid + ",'" + sname + "','"
+ saddr + "')";
         s.executeUpdate(insertQuery);

         System.out.println("Data inserted successfully into student table");

      } catch (SQLException err) {
         System.out.println("ERROR: " + err);
      }
   }
}
```

Output :

```
Inserting Data into student table:
        Enter student id:           101
        Enter student name:         John Doe
        Enter student address:      123 Main Street
        Data inserted successfully into student table
```

Step 3: Updating table data using Java Program

UpdateData.java

```
import java.sql.*;
import java.util.Scanner;

public class UpdateData {
   public static void main(String[] args) {
      try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/mydb", "root", "");
         Statements = con.createStatement()) {

         Scanner sc = new Scanner(System.in);
         System.out.println("Update Data in student table:");
         System.out.println("_____________________________________");

         System.out.print("Enter student id: ");
         int sid = sc.nextInt();
         System.out.print("Enter student name: ");
         String sname = sc.next();
         System.out.print("Enter student address: ");
         String saddr = sc.next();

         String updateQuery = "UPDATE student SET s_name='" + sname + "',
s_address='" + saddr + "' WHERE s_id=" + sid;
         s.executeUpdate(updateQuery);

         System.out.println("Data updated successfully");

      } catch (SQLException err) {
         System.out.println("ERROR: " + err);
      }
   }
}
```

Output :

Update Data in student table:

```
Enter student id:          101
Enter student name:        Jane Doe
Enter student address:     456 Broad Street
Data updated successfully
```

Step 4: Updating table data using Java Program

DeleteData.java

```
import java.sql.*;
import java.util.Scanner;

public class DeleteData {
   public static void main(String[] args) {
      try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost/mydb",
"root", "");
          Statements = con.createStatement()) {

         Scanner sc = new Scanner(System.in);
         System.out.println("Delete Data from student table:");
         System.out.println("_____");

         System.out.print("Enter student id: ");
         int sid = sc.nextInt();

         String deleteQuery = "DELETE FROM student WHERE s_id=" + sid;
         s.executeUpdate(deleteQuery);

         System.out.println("Data deleted successfully");

      } catch (SQLException err) {
         System.out.println("ERROR: " + err);
      }
   }
}
```

Output :

```
Delete Data from student table:
Enter student id:        101
Data deleted successfully
```

DisplayData.java

```
import java.sql.*;

public class DisplayData {
   public static void main(String[] args) {
      try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost/mydb",
"root", "");
         Statement s = con.createStatement()) {

         ResultSet rs = s.executeQuery("SELECT * FROM student");
         if (rs != null) {
            System.out.println("SID \t STU_NAME \t ADDRESS");
            System.out.println("_____");

            while (rs.next()) {
               System.out.println(rs.getString("s_id") + " \t " + rs.getString("s_name") + " \t " +
rs.getString("s_address"));
               System.out.println("_____");
            }
         }

      } catch (SQLException err) {
         System.out.println("ERROR: " + err);
      }
   }
}
```

Output :

```
SID   STU_NAME        ADDRESS
102    Alice Smith      789 Oak Avenue
103    Bob Johnson    567 Pine Road
```

6. **Create an xml for the bookstore. Validate the same using both DTD and XSD**.

bookstore.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">
<bookstore>
  <book>
    <title>Harry Potter and the Sorcerer's Stone</title>
    <author>J.K. Rowling</author>
    <genre>Fantasy</genre>
    <price>19.99</price>
  </book>
  <book>
    <title>To Kill a Mockingbird</title>
    <author>Harper Lee</author>
    <genre>Classic</genre>
    <price>14.99</price>
  </book>
  <book>
    <title>The Hobbit</title>
    <author>J.R.R. Tolkien</author>
    <genre>Fantasy</genre>
    <price>12.99</price>
  </book>
  <book>
    <title>1984</title>
    <author>George Orwell</author>
    <genre>Dystopian</genre>
    <price>13.99</price>
  </book>
  <book>
    <title>Pride and Prejudice</title>
    <author>Jane Austen</author>
    <genre>Classic</genre>
    <price>11.99</price>
  </book>
</bookstore>
```

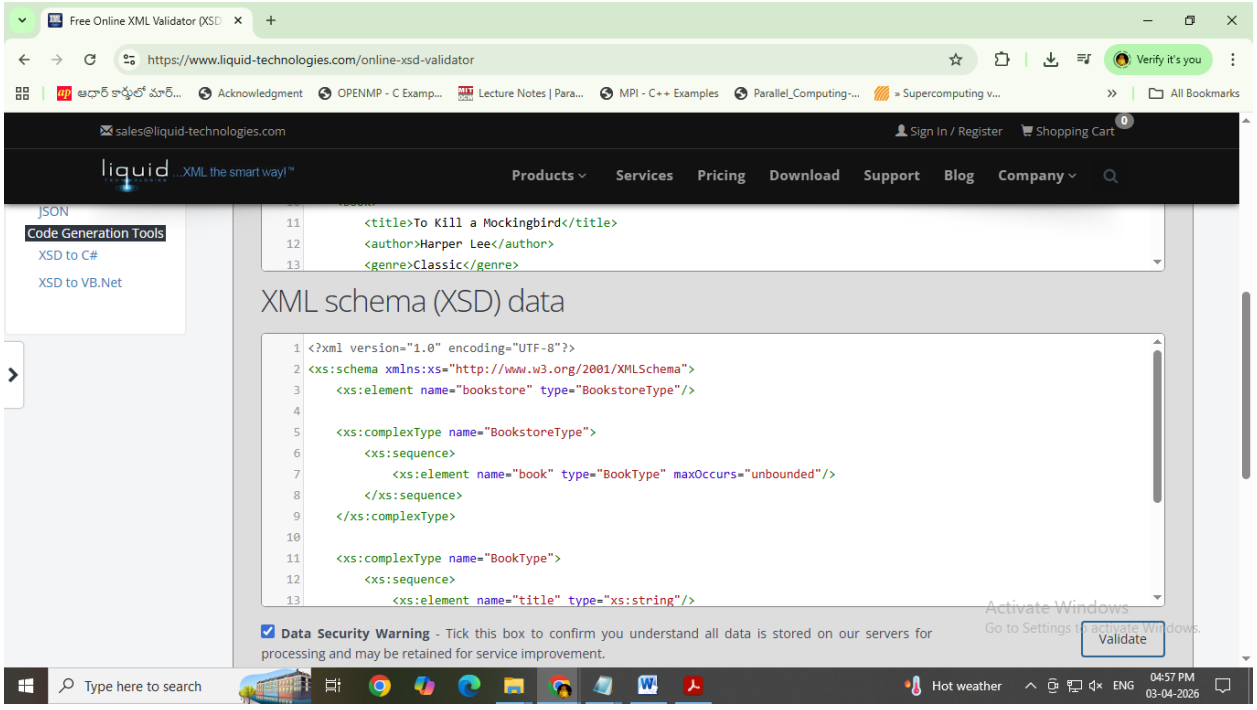
bookstore.dtd

```
<!ELEMENT bookstore (book+)>
<!ELEMENT book (title, author, genre, price)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT genre (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

bookstore.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="bookstore" type="BookstoreType"/>

  <xs:complexType name="BookstoreType">
    <xs:sequence>
      <xs:element name="book" type="BookType" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="BookType">
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
      <xs:element name="genre" type="xs:string"/>
      <xs:element name="price" type="xs:decimal"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Output:



XML schema (XSD) data

7. **Design a controller with servlet that provides the interaction with application developed in experiment 1 and the database created in experiment 5.**

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AppController extends HttpServlet {
   // Database credentials from Experiment 5
   private static final String DB_URL = "jdbc:mysql://localhost:3306/your_db";
   private static final String DB_USER = "root";
   private static final String DB_PASS = "password";

   protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      // 1. Retrieve data from Experiment 1 Form
      String name = request.getParameter("userName");
      String email = request.getParameter("email");

      try {
         // 2. Establish Connection
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);

         // 3. Execute DB Operation (Experiment 5 logic)
         String sql = "INSERT INTO USERDETAILS (name, email) VALUES (?, ?)";
         PreparedStatement pstmt = conn.prepareStatement(sql);
         pstmt.setString(1, name);
         pstmt.setString(2, email);
         int rows = pstmt.executeUpdate();

         // 4. Redirect to View
         if (rows > 0) {
            request.setAttribute("message", "Success!");
            RequestDispatcher rd = request.getRequestDispatcher("success.jsp");
            rd.forward(request, response);
         }
         conn.close();
      } catch (Exception e) {
         e.printStackTrace();
      }
   }
     }
```

8. **Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)**

(a) Using Cookies (Client-side storage)

server-cookie.js

```
const express = require('express');
const cookieParser = require('cookie-parser');
const app = express();

app.use(cookieParser());
app.use(express.urlencoded({ extended: true }));

// Add a new transaction
app.post('/add-transaction', (req, res) => {
  const action = req.body.action || 'Unknown action';
  let history = req.cookies.history ? JSON.parse(req.cookies.history) : [];

  history.push({
    timestamp: new Date().toISOString(),
    action: action
  });

  // Store as JSON string in cookie (maxAge = 30 minutes)
  res.cookie('history', JSON.stringify(history), { maxAge: 30 * 60 * 1000, httpOnly: true });
  res.send(`Transaction added: ${action}<br><a href="/history-cookie">View History</a>`);
});

// View history from cookie
app.get('/history-cookie', (req, res) => {
  const history = req.cookies.history ? JSON.parse(req.cookies.history) : [];
  let html = '<h2>Transaction History (Cookie)</h2><ul>';
  history.forEach(t => {
    html += `<li>${t.timestamp} - ${t.action}</li>`;
  });
  html += '</ul><a href="/">Add New Transaction</a>';
  res.send(html);
});

app.get('/', (req, res) => {
  res.send(`
    <h2>Add Transaction (Cookie Demo)</h2>
    <form method="POST" action="/add-transaction">
      <input type="text" name="action" placeholder="e.g. Purchased Laptop" required>
      <button type="submit">Add Transaction</button>
    </form>
    <a href="/history-cookie">View History</a>
  `);
});

app.listen(3000, () => console.log('Node.js Cookie Server running on
http://localhost:3000'));
```

Execution

```
npm init -y && npm install express cookie-parser
node server-cookie.js
```

(b) Using HTTP Session (Server-side with express-session)

server-session.js

```
const express = require('express');
const session = require('express-session');
const app = express();

app.use(session({
  secret: 'jntuh-lab-secret-key',
  resave: false,
  saveUninitialized: true,
  cookie: { maxAge: 30 * 60 * 1000 }  // 30 minutes
}));

app.use(express.urlencoded({ extended: true }));

app.post('/add-transaction-session', (req, res) => {
  const action = req.body.action || 'Unknown action';

  if (!req.session.history) req.session.history = [];

  req.session.history.push({
    timestamp: new Date().toISOString(),
    action: action
  });

  res.send(`Transaction added (Session): ${action}<br><a href="/history-session">View
History</a>`);
});

app.get('/history-session', (req, res) => {
  const history = req.session.history || [];
  let html = '<h2>Transaction History (HTTP Session)</h2><ul>';
  history.forEach(t => html += `<li>${t.timestamp} - ${t.action}</li>`);
  html += '</ul><a href="/">Add New Transaction</a>';
  res.send(html);
});

app.get('/', (req, res) => {
  res.send(`
    <h2>Add Transaction (Session Demo)</h2>
    <form method="POST" action="/add-transaction-session">
      <input type="text" name="action" placeholder="e.g. Added to Cart" required>
      <button type="submit">Add Transaction</button>
    </form>
    <a href="/history-session">View History</a>
  `);
});

app.listen(3001, () => console.log('Node.js Session Server running on http://localhost:3001'
```

Execution

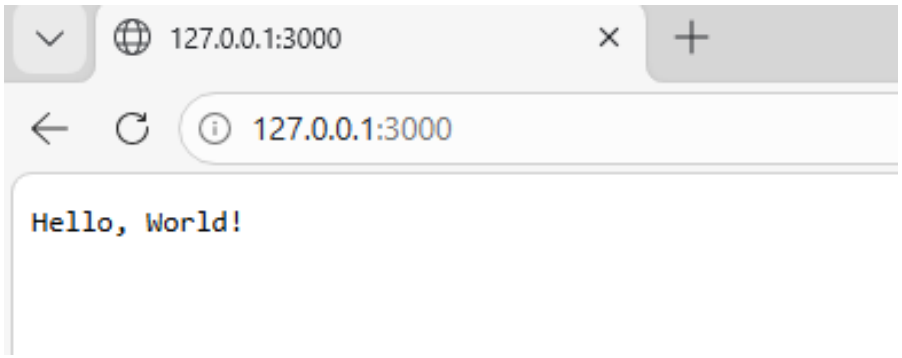
npm install express express-session then node server-session.js

9. **Create a custom server using http module and explore the other modules of Node JS like OS, path, event**

```
// Step 1: Import required modules
const http = require('http');
const os = require('os');
const path = require('path');
const { EventEmitter } = require('events');

// Step 2: Create an instance of EventEmitter
const eventEmitter = new EventEmitter();

// Step 3: Create a simple HTTP server
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, World!\n');
});

// Step 4: Define server port and hostname
const port = 3000;          // Fixed: Added space between const and port
const hostname = '127.0.0.1'; // Fixed: Added space between const and hostname

// Step 5: Listen for requests on the specified port and hostname
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`); // Fixed: Changed to backticks
});

// Step 6: Print OS information
console.log('OS Type:', os.type());
console.log('OS Platform:', os.platform());
console.log('OS Architecture:', os.arch());
console.log('CPU Cores:', os.cpus().length);

// Step 7: Print current working directory
console.log('Current Working Directory:', process.cwd());

// Step 8: Join paths using the path module
const joinedPath = path.join(__dirname, 'public', 'images');
console.log('Joined Path:', joinedPath);

// Step 9: Handle a custom event
eventEmitter.on('customEvent', (data) => {
  console.log('Custom Event Triggered:', data);
});

// Step 10: Emit a custom event
   eventEmitter.emit('customEvent', { message: 'Hello from custom event!' });
```

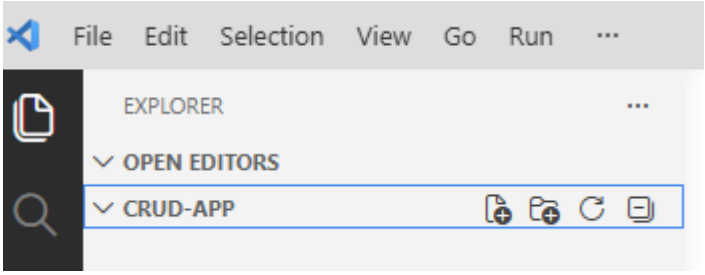
Output:

```
C:\Windows\System32\cmd.exe - node ex9.js

I:\NRDLab\weather\node progs>node ex9.js
OS Type: Windows_NT
OS Platform: win32
OS Architecture: x64
CPU Cores: 4
Current Working Directory: I:\NRDLab\weather\node progs
Joined Path: I:\NRDLab\weather\node progs\public\images
Custom Event Triggered: { message: 'Hello from custom event!' }
Server running at http://127.0.0.1:3000/
```

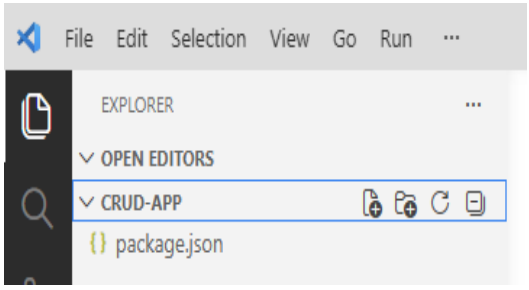


10. Develop an express web application that can interact with REST API to perform CRUD operations on student data.

Step 1: create a new folder and open the folder crud-app



Step 2: Change to the folder and execute the following command
npm init –y




```
I:\NRDLab\weather\crud-app>npm init -y
Wrote to I:\NRDLab\weather\crud-app\package.json:

{
  "name": "crud-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 3: Install express and sqlite3 packages using the following command

npm install express sqlite3

```
I:\NRDLab\weather\crud-app>npm install express sqlite3
npm warn deprecated prebuild-install@7.1.3: No longer maintained. Please contact the author of the relevant native addon;
alternatives are available.

added 151 packages, and audited 152 packages in 42s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

I:\NRDLab\weather\crud-app>
```

Step 4: Create two files named app.js and db.js in the crud-app folder

db.js

```
const sqlite3 = require('sqlite3').verbose();

// Global database instance
let db = null;

// Function to initialize the database schema
function initializeDatabase() {
```

```
  return new Promise((resolve, reject) => {
    db = new sqlite3.Database('./mydatabase.db', (err) => {
      if (err) {
        console.error('Database connection error:', err.message);
        reject(err);
      } else {
        console.log('Connected to the SQLite database.');
        createStudentsTable(db)
          .then(() => {
            console.log('Database initialization complete');
            resolve(db);
          })
          .catch(reject);
      }
    });
  });
}

// Function to create the 'students' table if it doesn't exist
function createStudentsTable(db) {
  return new Promise((resolve, reject) => {
    const createTableQuery = `
      CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER CHECK(age > 0 AND age < 150),
        grade TEXT CHECK(grade IN ('A', 'B', 'C', 'D', 'F')),
        email TEXT UNIQUE,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `;

    db.run(createTableQuery, (err) => {
      if (err) {
        console.error('Error creating students table:', err.message);
        reject(err);
      } else {
        console.log('The students table has been created or already exists.');

        // Create indexes for better performance
        createIndexes(db).then(resolve).catch(reject);
      }
    });
  });
}

// Function to create indexes
function createIndexes(db) {
  return new Promise((resolve, reject) => {
    const createIndexQuery = `
      CREATE INDEX IF NOT EXISTS idx_students_name ON students(name);
      CREATE INDEX IF NOT EXISTS idx_students_grade ON students(grade);
    `;
```

```
        db.exec(createIndexQuery, (err) => {
            if (err) {
                console.error('Error creating indexes:', err.message);
                reject(err);
            } else {
                console.log('Indexes created successfully');
                resolve();
            }
        });
    });
}

// Function to close database connection gracefully
function closeDatabase() {
    return new Promise((resolve, reject) => {
        if (db) {
            db.close((err) => {
                if (err) {
                    console.error('Error closing database:', err.message);
                    reject(err);
                } else {
                    console.log('Disconnected from the SQLite database.');
                    db = null;
                    resolve();
                }
            });
        } else {
            resolve();
        }
    });
}

// Enhanced version with CRUD operations
class StudentDatabase {
    constructor() {
        this.db = null;
    }

    async initialize() {
        try {
            this.db = await initializeDatabase();
            return this;
        } catch (error) {
            throw new Error(`Failed to initialize database: ${error.message}`);
        }
    }

    // Create a new student
    async createStudent(student) {
        return new Promise((resolve, reject) => {
            const { name, age, grade, email } = student;
            const insertQuery = `
                INSERT INTO students (name, age, grade, email)
```

```
            VALUES (?, ?, ?, ?)
        `;

        this.db.run(insertQuery, [name, age, grade, email], function(err) {
            if (err) {
                reject(err);
            } else {
                resolve({
                    id: this.lastID,
                    ...student
                });
            }
        });
    });
}

// Get all students
async getAllStudents() {
    return new Promise((resolve, reject) => {
        const selectQuery = 'SELECT * FROM students ORDER BY id DESC';

        this.db.all(selectQuery, (err, rows) => {
            if (err) {
                reject(err);
            } else {
                resolve(rows);
            }
        });
    });
}

// Get student by ID
async getStudentById(id) {
    return new Promise((resolve, reject) => {
        const selectQuery = 'SELECT * FROM students WHERE id = ?';

        this.db.get(selectQuery, [id], (err, row) => {
            if (err) {
                reject(err);
            } else {
                resolve(row);
            }
        });
    });
}

// Update student
async updateStudent(id, updates) {
    return new Promise((resolve, reject) => {
        const fields = [];
        const values = [];

        if (updates.name) {
            fields.push('name = ?');
```

```
            values.push(updates.name);
        }
        if (updates.age) {
            fields.push('age = ?');
            values.push(updates.age);
        }
        if (updates.grade) {
            fields.push('grade = ?');
            values.push(updates.grade);
        }
        if (updates.email) {
            fields.push('email = ?');
            values.push(updates.email);
        }

        fields.push('updated_at = CURRENT_TIMESTAMP');

        const updateQuery = `
            UPDATE students
            SET ${fields.join(', ')}
            WHERE id = ?
        `;
        values.push(id);

        this.db.run(updateQuery, values, function(err) {
            if (err) {
                reject(err);
            } else {
                resolve({
                    changes: this.changes,
                    id: id
                });
            }
        });
    });
}

// Delete student
async deleteStudent(id) {
    return new Promise((resolve, reject) => {
        const deleteQuery = 'DELETE FROM students WHERE id = ?';

        this.db.run(deleteQuery, [id], function(err) {
            if (err) {
                reject(err);
            } else {
                resolve({
                    deleted: this.changes > 0,
                    id: id
                });
            }
        });
    });
}
```

```
// Search students by name
async searchStudentsByName(name) {
  return new Promise((resolve, reject) => {
    const searchQuery = 'SELECT * FROM students WHERE name LIKE ? ORDER BY name';

    this.db.all(searchQuery, [`%${name}%`], (err, rows) => {
      if (err) {
        reject(err);
      } else {
        resolve(rows);
      }
    });
  });
}

// Get students by grade
async getStudentsByGrade(grade) {
  return new Promise((resolve, reject) => {
    const gradeQuery = 'SELECT * FROM students WHERE grade = ? ORDER BY name';

    this.db.all(gradeQuery, [grade], (err, rows) => {
      if (err) {
        reject(err);
      } else {
        resolve(rows);
      }
    });
  });
}

// Get statistics
async getStatistics() {
  return new Promise((resolve, reject) => {
    const statsQuery = `
      SELECT
        COUNT(*) as total_students,
        AVG(age) as average_age,
        MIN(age) as youngest_age,
        MAX(age) as oldest_age,
        grade,
        COUNT(*) as count_per_grade
      FROM students
      GROUP BY grade
    `;

    this.db.all(statsQuery, (err, rows) => {
      if (err) {
        reject(err);
      } else {
        resolve(rows);
      }
    });
  });
```

```
    }

    // Close database connection
    async close() {
        return await closeDatabase();
    }
}

// Example usage with async/await
async function main() {
    const studentDB = new StudentDatabase();

    try {
        // Initialize database
        await studentDB.initialize();
        console.log('Database ready!');

        // Create a new student
        const newStudent = await studentDB.createStudent({
            name: 'John Doe',
            age: 20,
            grade: 'A',
            email: 'john.doe@example.com'
        });
        console.log('Student created:', newStudent);

        // Create another student
        await studentDB.createStudent({
            name: 'Jane Smith',
            age: 22,
            grade: 'B',
            email: 'jane.smith@example.com'
        });

        // Get all students
        const allStudents = await studentDB.getAllStudents();
        console.log(`\n    Total students: ${allStudents.length}`);
        console.log('All students:', allStudents);

        // Get student by ID
        const student = await studentDB.getStudentById(1);
        console.log('\n    Student with ID 1:', student);

        // Update student
        const updated = await studentDB.updateStudent(1, {
            grade: 'A+',
            age: 21
        });
        console.log('\n✏ Update result:', updated);

        // Search students
        const searchResults = await studentDB.searchStudentsByName('John');
        console.log('\n    Search results:', searchResults);
```

```
        // Get statistics
        const stats = await studentDB.getStatistics();
        console.log('\n    Statistics:', stats);

    } catch (error) {
        console.error(' Error:', error.message);
    } finally {
        // Close database connection
        await studentDB.close();
        console.log('    Database connection closed');
    }
}

// Handle process termination gracefully
process.on('SIGINT', async () => {
    console.log('\n\n□  Received SIGINT. Closing database...');
    await closeDatabase();
    process.exit(0);
});

process.on('SIGTERM', async () => {
    console.log('\n\n□  Received SIGTERM. Closing database...');
    await closeDatabase();
    process.exit(0);
});

// Export the enhanced class and functions
module.exports = {
    initializeDatabase,
    closeDatabase,
    StudentDatabase
};

// Run the example if this script is executed directly
if (require.main === module) {
    main().catch(console.error);
}
```

app.js

```
const express = require('express');
const sqlite3 = require('sqlite3').verbose(); // Fixed: Added .verbose()
const { initializeDatabase } = require('./db');
const app = express();
const port = 3000;

// Connect to SQLite database
const db = new sqlite3.Database('./mydatabase.db', (err) => {
    if (err) {
        console.error('Database connection error:', err.message); // Fixed: Changed log to error
    } else {
        console.log('Connected to the SQLite database.');
        // Initialize database schema if needed
        initializeDatabase().catch(console.error);
    }
```

```
});

// Middleware to parse request body as JSON
app.use(express.json());

// Root endpoint
app.get('/', (req, res) => {
  res.send('Welcome to the Student Management System'); // Fixed: Complete message
});

// Get all students
app.get('/students', (req, res) => {
  db.all('SELECT * FROM students ORDER BY id DESC', [], (err, rows) => {
    if (err) {
      console.error('Error fetching students:', err.message);
      return res.status(500).json({ error: 'Internal server error' });
    }
    res.json({
      success: true,
      count: rows.length,
      students: rows
    });
  });
});

// Get a single student by id
app.get('/students/:id', (req, res) => {
  const id = req.params.id;

  // Validate ID
  if (isNaN(id) || id <= 0) {
    return res.status(400).json({ error: 'Invalid student ID' });
  }

  db.get('SELECT * FROM students WHERE id = ?', [id], (err, row) => { // Fixed: Changed to
db.get()
    if (err) {
      console.error('Error fetching student:', err.message);
      return res.status(500).json({ error: 'Internal server error' });
    }

    if (!row) {
      return res.status(404).json({ error: `Student with ID ${id} not found` });
    }

    res.json({
      success: true,
      student: row
    });
  });
});

// Create a new student
app.post('/students', (req, res) => {
```

```
    const { name, age, grade, email } = req.body;

    // Input validation
    if (!name || !age || !grade) {
      return res.status(400).json({
        error: 'Missing required fields',
        required: ['name', 'age', 'grade']
      });
    }

    // Validate age
    if (isNaN(age) || age < 1 || age > 150) {
      return res.status(400).json({ error: 'Age must be between 1 and 150' });
    }

    // Validate grade
    const validGrades = ['A', 'B', 'C', 'D', 'F', 'A+', 'A-', 'B+', 'B-'];
    if (!validGrades.includes(grade.toUpperCase())) {
      return res.status(400).json({ error: 'Invalid grade value' });
    }

    const insertQuery = `INSERT INTO students (name, age, grade, email)
                         VALUES (?, ?, ?, ?)`;

    db.run(insertQuery, [name, age, grade.toUpperCase(), email || null], function(err) {
      if (err) {
        console.error('Error creating student:', err.message);

        // Handle unique constraint violation
        if (err.message.includes('UNIQUE')) {
          return res.status(409).json({ error: 'Email already exists' });
        }

        return res.status(500).json({ error: 'Internal server error' });
      }

      // Fetch the newly created student
      db.get('SELECT * FROM students WHERE id = ?', [this.lastID], (err, row) => {
        if (err) {
          return res.status(201).json({
            success: true,
            id: this.lastID,
            message: 'Student created successfully'
          });
        }

        res.status(201).json({
          success: true,
          message: 'Student created successfully',
          student: row
        });
      });
    });
  });
```

```
// Update a student
app.put('/students/:id', (req, res) => {
  const id = req.params.id;
  const { name, age, grade, email } = req.body;

  // Validate ID
  if (isNaN(id) || id <= 0) {
    return res.status(400).json({ error: 'Invalid student ID' });
  }

  // Check if student exists
  db.get('SELECT * FROM students WHERE id = ?', [id], (err, existingStudent) => {
    if (err) {
      console.error('Error checking student:', err.message);
      return res.status(500).json({ error: 'Internal server error' });
    }

    if (!existingStudent) {
      return res.status(404).json({ error: `Student with ID ${id} not found` });
    }

    // Validate age if provided
    if (age && (isNaN(age) || age < 1 || age > 150)) {
      return res.status(400).json({ error: 'Age must be between 1 and 150' });
    }

    // Validate grade if provided
    if (grade) {
      const validGrades = ['A', 'B', 'C', 'D', 'F', 'A+', 'A-', 'B+', 'B-'];
      if (!validGrades.includes(grade.toUpperCase())) {
        return res.status(400).json({ error: 'Invalid grade value' });
      }
    }

    // Build dynamic update query
    const updates = [];
    const values = [];

    if (name) {
      updates.push('name = ?');
      values.push(name);
    }
    if (age) {
      updates.push('age = ?');
      values.push(age);
    }
    if (grade) {
      updates.push('grade = ?');
      values.push(grade.toUpperCase());
    }
    if (email) {
      updates.push('email = ?');
      values.push(email);
```

```
        }

        updates.push('updated_at = CURRENT_TIMESTAMP');
        values.push(id);

        const updateQuery = `UPDATE students SET ${updates.join(', ')} WHERE id = ?`;

        db.run(updateQuery, values, function(err) {
          if (err) {
            console.error('Error updating student:', err.message);

            if (err.message.includes('UNIQUE')) {
              return res.status(409).json({ error: 'Email already exists' });
            }

            return res.status(500).json({ error: 'Internal server error' });
          }

          // Fetch updated student
          db.get('SELECT * FROM students WHERE id = ?', [id], (err, updatedStudent) => {
            if (err) {
              return res.json({
                success: true,
                message: 'Student updated successfully',
                updatedId: id
              });
            }

            res.json({
              success: true,
              message: 'Student updated successfully',
              student: updatedStudent
            });
          });
        });
      });
    });

    // Delete a student
    app.delete('/students/:id', (req, res) => {
      const id = req.params.id;

      // Validate ID
      if (isNaN(id) || id <= 0) {
        return res.status(400).json({ error: 'Invalid student ID' });
      }

      // Check if student exists
      db.get('SELECT * FROM students WHERE id = ?', [id], (err, existingStudent) => {
        if (err) {
          console.error('Error checking student:', err.message);
          return res.status(500).json({ error: 'Internal server error' });
        }
```

```
        if (!existingStudent) {
            return res.status(404).json({ error: `Student with ID ${id} not found` });
        }

        db.run('DELETE FROM students WHERE id = ?', [id], function(err) {
            if (err) {
                console.error('Error deleting student:', err.message);
                return res.status(500).json({ error: 'Internal server error' });
            }

            res.json({
                success: true,
                message: 'Student deleted successfully',
                deletedId: id,
                student: existingStudent
            });
        });
    });
});

// Additional endpoints for better functionality

// Get students by grade
app.get('/students/grade/:grade', (req, res) => {
    const grade = req.params.grade;

    db.all('SELECT * FROM students WHERE grade = ? ORDER BY name',
[grade.toUpperCase()], (err, rows) => {
        if (err) {
            console.error('Error fetching students by grade:', err.message);
            return res.status(500).json({ error: 'Internal server error' });
        }

        res.json({
            success: true,
            grade: grade,
            count: rows.length,
            students: rows
        });
    });
});

// Search students by name
app.get('/students/search/:name', (req, res) => {
    const name = req.params.name;

    db.all('SELECT * FROM students WHERE name LIKE ? ORDER BY name', [`%${name}%`],
(err, rows) => {
        if (err) {
            console.error('Error searching students:', err.message);
            return res.status(500).json({ error: 'Internal server error' });
        }

        res.json({
```

```
            success: true,
            searchTerm: name,
            count: rows.length,
            students: rows
        });
    });
});

// Get statistics
app.get('/stats', (req, res) => {
    const statsQuery = `
        SELECT
            COUNT(*) as total_students,
            AVG(age) as average_age,
            MIN(age) as youngest_age,
            MAX(age) as oldest_age,
            COUNT(DISTINCT grade) as distinct_grades
        FROM students
    `;

    db.get(statsQuery, [], (err, stats) => {
        if (err) {
            console.error('Error fetching statistics:', err.message);
            return res.status(500).json({ error: 'Internal server error' });
        }

        // Get grade distribution
        db.all('SELECT grade, COUNT(*) as count FROM students GROUP BY grade', [], (err,
gradeDist) => {
            if (err) {
                return res.json({
                    success: true,
                    statistics: stats
                });
            }

            res.json({
                success: true,
                statistics: stats,
                gradeDistribution: gradeDist
            });
        });
    });
});

// Error handling middleware
app.use((err, req, res, next) => {
    console.error('Unhandled error:', err);
    res.status(500).json({
        error: 'Something went wrong!',
        message: process.env.NODE_ENV === 'development' ? err.message : undefined
    });
});
```

```
// 404 handler for undefined routes
app.use((req, res) => {
   res.status(404).json({
      error: 'Route not found',
      message: `Cannot ${req.method} ${req.url}`
   });
});

// Start server
app.listen(port, () => {
   console.log(`Server running at http://localhost:${port}`); // Fixed: Changed to backticks
   console.log(`Available endpoints:`);
   console.log(`  GET    /students          - Get all students`);
   console.log(`  GET    /students/:id      - Get student by ID`);
   console.log(`  GET    /students/grade/:grade - Get students by grade`);
   console.log(`  GET    /students/search/:name - Search students by name`);
   console.log(`  POST   /students          - Create new student`);
   console.log(`  PUT    /students/:id      - Update student`);
   console.log(`  DELETE /students/:id      - Delete student`);
   console.log(`  GET    /stats             - Get statistics`);
});

// Graceful shutdown
process.on('SIGINT', () => {
   console.log('\nShutting down server...');
   db.close((err) => {
      if (err) {
         console.error('Error closing database:', err.message);
      } else {
         console.log('Database connection closed');
      }
      process.exit(0);
   });
});
```

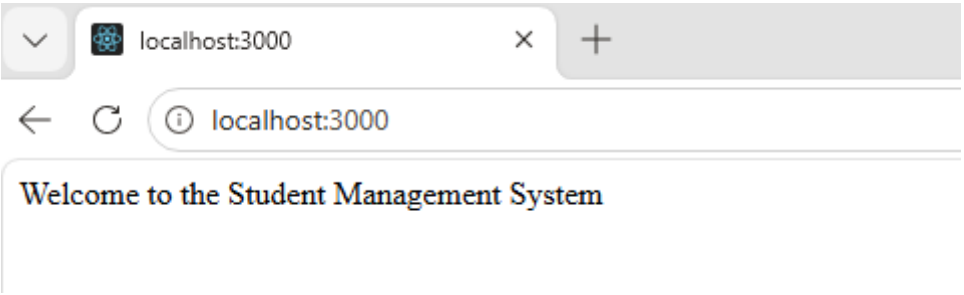
Step 5: Execute **db.js** to create the database *mydatabase.db*

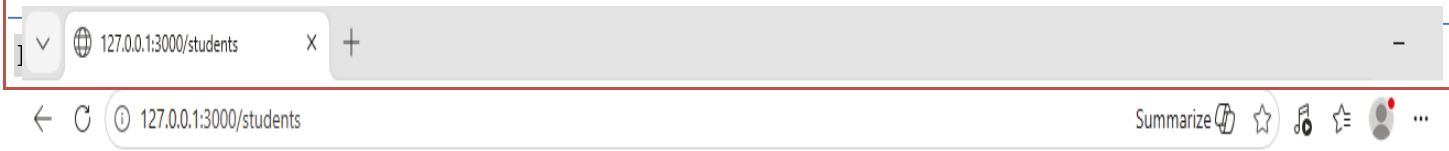
Step 6: Execute app.js

Step 7: Open the browser enter the below url
http://127.0.0.1:3000/students

```
I:\NRDLab\weather\crud-app>node db.js
Connected to the SQLite database.
The students table has been created or already exists.
Indexes created successfully
Database initialization complete
📊 Database ready!
📊 Student created: {
  id: 1,
  name: 'John Doe',
  age: 20,
  grade: 'A',
  email: 'john.doe@example.com'
}

📊  Total students: 2
All students: [
  {
    id: 2,
    name: 'Jane Smith',
    age: 22,
    grade: 'B',
    email: 'jane.smith@example.com',
    created_at: '2026-04-03 12:47:08',
    updated_at: '2026-04-03 12:47:08'
  },
  {
    id: 1,
    name: 'John Doe',
    age: 20,
    grade: 'A',
    email: 'john.doe@example.com',
    created_at: '2026-04-03 12:47:08',
    updated_at: '2026-04-03 12:47:08'
  }
```

```
I:\NRDLab\weather\crud-app>node app.js
Server running at http://localhost:3000
Available endpoints:
  GET    /students            - Get all students
  GET    /students/:id        - Get student by ID
  GET    /students/grade/:grade - Get students by grade
  GET    /students/search/:name - Search students by name
  POST   /students            - Create new student
  PUT    /students/:id        - Update student
  DELETE /students/:id        - Delete student
  GET    /stats               - Get statistics
Connected to the SQLite database.
Connected to the SQLite database.
The students table has been created or already exists.
Indexes created successfully
Database initialization complete
```





11. For the above application create authorized end points using JWT (JSON Web Token).

Step 1: Project Setup

```
mkdir student-crud-jwt
cd student-crud-jwt
npm init -y
npm install express jsonwebtoken bcryptjs dotenv
npm install -D nodemon
```

```
student-crud-jwt/
├── .env
├── server.js
├── middleware/
│   └── auth.js
├── routes/
│   ├── auth.js
│   └── students.js
└── data/
    └── students.js
```

Step 2. .env file

```
PORT=5000
JWT_SECRET=jntuh_lab_jwt_secret_key_2026   # Change this in production
```

Step 3:

```
let students = [
  { id: 1, name: "Sreeni", rollNo: "21B01A05A1", branch: "CSE", year: 3 },
  { id: 2, name: "Rahul", rollNo: "21B01A05A2", branch: "IT", year: 2 }
];

let nextId = 3;

module.exports = { students, nextId };
```

Step 4. middleware/auth.js (JWT Authentication Middleware)

```
JavaScriptconst jwt = require('jsonwebtoken');

const authenticateJWT = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Access Denied: No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;       // Attach user info to request
    next();
  } catch (err) {
    return res.status(403).json({ message: 'Invalid or Expired Token' });
```

```
  }
};

module.exports = authenticateJWT;
```

Step 5. routes/auth.js (Registration & Login)

JavaScript

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const router = express.Router();

const users = [];   // In-memory users (for lab)

// Register User
router.post('/register', async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({ message: 'Username and password required' });
  }

  const existingUser = users.find(u => u.username === username);
  if (existingUser) {
    return res.status(400).json({ message: 'User already exists' });
  }

  const hashedPassword = await bcrypt.hash(password, 10);

  users.push({ username, password: hashedPassword });
  res.status(201).json({ message: 'User registered successfully' });
});

// Login User
router.post('/login', async (req, res) => {
  const { username, password } = req.body;

  const user = users.find(u => u.username === username);
  if (!user) {
    return res.status(400).json({ message: 'Invalid credentials' });
  }

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) {
    return res.status(400).json({ message: 'Invalid credentials' });
  }

  // Generate JWT Token (valid for 1 hour)
  const token = jwt.sign(
    { username: user.username },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );

  res.json({
```

```
    message: 'Login successful',
    token
  });
});

module.exports = router;
```

Step 6. routes/students.js (Protected CRUD Routes)

```
JavaScriptconst express = require('express');
const router = express.Router();
const authenticateJWT = require('../middleware/auth');
const { students, nextId } = require('../data/students');

// All routes below are protected by JWT
router.use(authenticateJWT);

// Get all students
router.get('/', (req, res) => {
  res.json(students);
});

// Get single student by ID
router.get('/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) return res.status(404).json({ message: 'Student not found' });
  res.json(student);
});

// Create new student
router.post('/', (req, res) => {
  const { name, rollNo, branch, year } = req.body;

  if (!name || !rollNo || !branch || !year) {
    return res.status(400).json({ message: 'All fields are required' });
  }

  const newStudent = {
    id: nextId++,
    name,
    rollNo,
    branch,
    year: parseInt(year)
  };

  students.push(newStudent);
  res.status(201).json(newStudent);
});

// Update student
router.put('/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) return res.status(404).json({ message: 'Student not found' });

  const { name, rollNo, branch, year } = req.body;
```

```
  if (name) student.name = name;
  if (rollNo) student.rollNo = rollNo;
  if (branch) student.branch = branch;
  if (year) student.year = parseInt(year);

  res.json({ message: 'Student updated', student });
});

// Delete student
router.delete('/:id', (req, res) => {
  const index = students.findIndex(s => s.id === parseInt(req.params.id));
  if (index === -1) return res.status(404).json({ message: 'Student not found' });

  students.splice(index, 1);
  res.json({ message: 'Student deleted successfully' });
});

module.exports = router;
```

Step 7. server.js (Main File)

```
JavaScriptrequire('dotenv').config();
const express = require('express');
const cors = require('cors');   // Optional but recommended

const app = express();
const PORT = process.env.PORT || 5000;

app.use(express.json());
app.use(cors());

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/students', require('./routes/students'));

// Home route
app.get('/', (req, res) => {
  res.send(`
    <h2>Student CRUD API with JWT Authentication</h2>
    <p>Use Postman to test the following endpoints:</p>
    <ul>
      <li>POST /api/auth/register</li>
      <li>POST /api/auth/login</li>
      <li>All /api/students routes require Authorization: Bearer &lt;token&gt;</li>
    </ul>
  `);
});

app.listen(PORT, () => {
  console.log(`    Server running on http://localhost:${PORT}`);
  console.log(`Test with Postman!`);
});
```

Execution:

Add "start": "nodemon server.js" in package.json scripts.
npm start

Postman Testing Guide
1. Register a user

POST → http://localhost:5000/api/auth/register
Body (JSON):
JSON
```
{
  "username": "sreeni",
  "password": "123456"
}
```

http://localhost:5000/api/auth/login

12. Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Management System</title>
  <script src="https://unpkg.com/react@18/umd/react.production.min.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.production.min.js"></script>
  <script src="https://unpkg.com/react-router-dom@6/umd/react-router-dom.production.min.js"></script>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      color: #333;
    }
    .navbar {
      background: rgba(255, 255, 255, 0.95);
      backdrop-filter: blur(10px);
      box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
      padding: 1rem 2rem;
      position: sticky;
      top: 0;
      z-index: 100;
    }
    .nav-container {
      max-width: 1200px;
      margin: 0 auto;
      display: flex;
      align-items: center;
      justify-content: space-between;
    }
    .logo {
      font-size: 1.8rem;
      font-weight: 800;
      background: linear-gradient(90deg, #667eea, #764ba2);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
    }
    .nav-links {
      display: flex;
      gap: 2rem;
      list-style: none;
    }
    .nav-links a {
      text-decoration: none;
      color: #333;
```

```
    font-weight: 600;
    padding: 8px 16px;
    border-radius: 50px;
    transition: all 0.3s ease;
}
.nav-links a:hover, .nav-links a.active {
    background: #667eea;
    color: white;
    transform: translateY(-2px);
}
.main-content {
    max-width: 1200px;
    margin: 2rem auto;
    padding: 0 2rem;
}
.page {
    background: white;
    border-radius: 20px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.1);
    padding: 3rem;
    min-height: calc(100vh - 120px);
}
h1 {
    font-size: 2.8rem;
    margin-bottom: 1rem;
    background: linear-gradient(90deg, #667eea, #764ba2);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
.form-group {
    margin-bottom: 1.5rem;
}
label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 600;
    color: #555;
}
input, textarea, select {
    width: 100%;
    padding: 14px;
    border: 2px solid #e1e5e9;
    border-radius: 12px;
    font-size: 1rem;
    transition: all 0.3s ease;
}
input:focus, textarea:focus, select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 4px rgba(102, 126, 234, 0.2);
}
button {
    background: linear-gradient(90deg, #667eea, #764ba2);
    color: white;
```

```
            border: none;
            padding: 14px 32px;
            font-size: 1.1rem;
            font-weight: 600;
            border-radius: 50px;
            cursor: pointer;
            transition: all 0.3s ease;
            box-shadow: 0 8px 25px rgba(102, 126, 234, 0.3);
        }
        button:hover {
            transform: translateY(-3px);
            box-shadow: 0 12px 30px rgba(102, 126, 234, 0.4);
        }
        .student-table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 2rem;
        }
        .student-table th, .student-table td {
            padding: 1rem;
            text-align: left;
            border-bottom: 1px solid #eee;
        }
        .student-table th {
            background: #f8f9fa;
            color: #555;
            font-weight: 600;
        }
        .card {
            background: #f8f9fa;
            border-radius: 16px;
            padding: 2rem;
            margin-bottom: 2rem;
        }
    </style>
</head>
<body>
    <div id="root"></div>

    <script>
        const { createRoot } = ReactDOM;
        const { BrowserRouter: Router, Routes, Route, Link, NavLink, useNavigate } =
ReactRouterDOM;
        const { useState } = React;

        // Navbar Component
        function Navbar() {
            return React.createElement('nav', { className: 'navbar' },
                React.createElement('div', { className: 'nav-container' },
                    React.createElement('div', { className: 'logo' }, '    SMS'),
                    React.createElement('ul', { className: 'nav-links' },
                        React.createElement('li', null,
                            React.createElement(NavLink, {
                                to: '/',
```

```
                    className: ({ isActive }) => isActive ? 'active' : ''
                  }, 'Home')
                ),
                React.createElement('li', null,
                  React.createElement(NavLink, {
                    to: '/register',
                    className: ({ isActive }) => isActive ? 'active' : ''
                  }, 'Register')
                ),
                React.createElement('li', null,
                  React.createElement(NavLink, {
                    to: '/login',
                    className: ({ isActive }) => isActive ? 'active' : ''
                  }, 'Login')
                ),
                React.createElement('li', null,
                  React.createElement(NavLink, {
                    to: '/contact',
                    className: ({ isActive }) => isActive ? 'active' : ''
                  }, 'Contact')
                ),
                React.createElement('li', null,
                  React.createElement(NavLink, {
                    to: '/about',
                    className: ({ isActive }) => isActive ? 'active' : ''
                  }, 'About')
                )
              )
            )
          );
        }

        // Home Page - Dashboard with Student List
        function Home() {
          const [students, setStudents] = useState([
            { id: 1, name: 'Aarav Sharma', email: 'aarav@example.com', course: 'Computer
Science', grade: 'A' },
            { id: 2, name: 'Priya Patel', email: 'priya@example.com', course: 'Business
Administration', grade: 'B+' },
            { id: 3, name: 'Rahul Verma', email: 'rahul@example.com', course: 'Mechanical
Engineering', grade: 'A-' }
          ]);

          return React.createElement('div', { className: 'page' },
            React.createElement('h1', null, 'Welcome to Student Management System'),
            React.createElement('p', { style: { fontSize: '1.3rem', marginBottom: '2rem', color:
'#666' } },
              'Manage students, track progress, and streamline academic operations.'
            ),
            React.createElement('div', { className: 'card' },
              React.createElement('h2', null, '    Current Students'),
              React.createElement('table', { className: 'student-table' },
                React.createElement('thead', null,
                  React.createElement('tr', null,
```

```
                React.createElement('th', null, 'ID'),
                React.createElement('th', null, 'Name'),
                React.createElement('th', null, 'Email'),
                React.createElement('th', null, 'Course'),
                React.createElement('th', null, 'Grade')
              )
            ),
            React.createElement('tbody', null,
              students.map(student =>
                React.createElement('tr', { key: student.id },
                  React.createElement('td', null, student.id),
                  React.createElement('td', null, student.name),
                  React.createElement('td', null, student.email),
                  React.createElement('td', null, student.course),
                  React.createElement('td', null, student.grade)
                )
              )
            )
          )
        ),
        React.createElement('div', { style: { textAlign: 'center', marginTop: '3rem' } },
          React.createElement('p', { style: { fontSize: '1.1rem', color: '#666', marginBottom:
'1rem' } },
            'Total Students: ', students.length
          ),
          React.createElement(Link, {
            to: '/register',
            style: {
              display: 'inline-block',
              background: 'linear-gradient(90deg, #667eea, #764ba2)',
              color: 'white',
              padding: '16px 40px',
              borderRadius: '50px',
              textDecoration: 'none',
              fontWeight: 600,
              fontSize: '1.1rem'
            }
          }, 'Register New Student')
        )
      );
    }

    // Registration Page
    function Register() {
      const [formData, setFormData] = useState({
        fullName: '',
        email: '',
        studentId: '',
        course: 'Computer Science',
        password: ''
      });
      const navigate = useNavigate();

      const handleChange = (e) => {
```

```
        setFormData({
          ...formData,
          [e.target.name]: e.target.value
        });
      };

      const handleSubmit = (e) => {
        e.preventDefault();
        alert(`Student ${formData.fullName} registered successfully!\nStudent ID: ${formData.studentId}`);
        // In a real app, this would send data to backend
        navigate('/');
      };

      return React.createElement('div', { className: 'page' },
        React.createElement('h1', null, 'Student Registration'),
        React.createElement('form', { onSubmit: handleSubmit },
          React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Full Name'),
            React.createElement('input', {
              type: 'text',
              name: 'fullName',
              value: formData.fullName,
              onChange: handleChange,
              placeholder: 'Enter full name',
              required: true
            })
          ),
          React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Email Address'),
            React.createElement('input', {
              type: 'email',
              name: 'email',
              value: formData.email,
              onChange: handleChange,
              placeholder: 'student@university.edu',
              required: true
            })
          ),
          React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Student ID'),
            React.createElement('input', {
              type: 'text',
              name: 'studentId',
              value: formData.studentId,
              onChange: handleChange,
              placeholder: 'STU2026XXXX',
              required: true
            })
          ),
          React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Course'),
            React.createElement('select', {
              name: 'course',
```

```
                value: formData.course,
                onChange: handleChange
            },
                React.createElement('option', { value: 'Computer Science' }, 'Computer
Science'),
                React.createElement('option', { value: 'Business Administration' }, 'Business
Administration'),
                React.createElement('option', { value: 'Mechanical Engineering' }, 'Mechanical
Engineering'),
                React.createElement('option', { value: 'Electronics' }, 'Electronics &
Communication')
            )
        ),
        React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Password'),
            React.createElement('input', {
                type: 'password',
                name: 'password',
                value: formData.password,
                onChange: handleChange,
                placeholder: 'Create password',
                required: true
            })
        ),
        React.createElement('button', { type: 'submit', style: { width: '100%' } }, 'Register
Student')
    )
  );
}

// Login Page
function Login() {
  const [formData, setFormData] = useState({ email: '', password: '' });
  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(` Welcome back, ${formData.email.split('@')[0]}! You are now logged in.`);
    navigate('/');
  };

  return React.createElement('div', { className: 'page' },
    React.createElement('h1', null, 'Student / Admin Login'),
    React.createElement('form', { onSubmit: handleSubmit },
      React.createElement('div', { className: 'form-group' },
        React.createElement('label', null, 'Email / Student ID'),
        React.createElement('input', {
```

```
                type: 'text',
                name: 'email',
                value: formData.email,
                onChange: handleChange,
                placeholder: 'Enter email or student ID',
                required: true
            })
        ),
        React.createElement('div', { className: 'form-group' },
            React.createElement('label', null, 'Password'),
            React.createElement('input', {
                type: 'password',
                name: 'password',
                value: formData.password,
                onChange: handleChange,
                placeholder: 'Enter password',
                required: true
            })
        ),
        React.createElement('button', { type: 'submit', style: { width: '100%' } }, 'Login')
    ),
    React.createElement('p', { style: { textAlign: 'center', marginTop: '2rem', color:
'#666' } },
        "Don't have an account? ",
        React.createElement(Link, { to: '/register' }, 'Register here')
    )
  );
}

// Contact Page
function Contact() {
  const [formData, setFormData] = useState({ name: '', email: '', message: '' });

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(' Thank you! Your message has been sent to the administration.');
    setFormData({ name: '', email: '', message: '' });
  };

  return React.createElement('div', { className: 'page' },
    React.createElement('h1', null, 'Contact Us'),
    React.createElement('p', { style: { marginBottom: '2rem', color: '#666' } },
      'Have questions? Reach out to our support team.'
    ),
    React.createElement('form', { onSubmit: handleSubmit },
      React.createElement('div', { className: 'form-group' },
        React.createElement('label', null, 'Your Name'),
```

```
                React.createElement('input', {
                    type: 'text',
                    name: 'name',
                    value: formData.name,
                    onChange: handleChange,
                    required: true
                })
            ),
            React.createElement('div', { className: 'form-group' },
                React.createElement('label', null, 'Email Address'),
                React.createElement('input', {
                    type: 'email',
                    name: 'email',
                    value: formData.email,
                    onChange: handleChange,
                    required: true
                })
            ),
            React.createElement('div', { className: 'form-group' },
                React.createElement('label', null, 'Message'),
                React.createElement('textarea', {
                    name: 'message',
                    value: formData.message,
                    onChange: handleChange,
                    rows: 6,
                    placeholder: 'How can we help you today?',
                    required: true
                })
            ),
            React.createElement('button', { type: 'submit', style: { width: '100%' } }, 'Send
Message')
        )
    );
}

// About Page
function About() {
    return React.createElement('div', { className: 'page' },
        React.createElement('h1', null, 'About Student Management System'),
        React.createElement('div', { className: 'card' },
            React.createElement('p', { style: { fontSize: '1.2rem', lineHeight: '1.8' } },
                'The Student Management System (SMS) is a modern web application designed
to simplify student administration, registration, and academic tracking.'
            ),
            React.createElement('h3', { style: { margin: '2rem 0 1rem' } }, 'Key Features'),
            React.createElement('ul', { style: { paddingLeft: '2rem', lineHeight: '2' } },
                React.createElement('li', null, ' Easy student registration and login'),
                React.createElement('li', null, ' Real-time student dashboard'),
                React.createElement('li', null, ' Secure authentication system'),
                React.createElement('li', null, 'Contact & support portal'),
                React.createElement('li', null, ' Responsive design for all devices')
            ),
            React.createElement('p', { style: { marginTop: '2rem', fontStyle: 'italic' } },
                'Built with  using React and React Router for seamless navigation.'
```

```
                )
            )
        );
    }

    // Main App Component
    function App() {
        return React.createElement(Router, null,
            React.createElement(React.Fragment, null,
                React.createElement(Navbar),
                React.createElement('div', { className: 'main-content' },
                    React.createElement(Routes, null,
                        React.createElement(Route, { path: '/', element: React.createElement(Home)
}),
                        React.createElement(Route, { path: '/register', element:
React.createElement(Register) }),
                        React.createElement(Route, { path: '/login', element:
React.createElement(Login) }),
                        React.createElement(Route, { path: '/contact', element:
React.createElement(Contact) }),
                        React.createElement(Route, { path: '/about', element:
React.createElement(About) })
                    )
                )
            )
        );
    }

    // Render the app
    const root = createRoot(document.getElementById('root'));
    root.render(
        React.createElement(React.StrictMode, null,
            React.createElement(App)
        )
    );
  </script>
</body>
</html>
```

13. Create a service in react that fetches the weather information from penweathermap.org and the display the current and historical weather information using graphical representation using chart.js

Step 1: Create the React App

```
npx create-react-app weather-app
cd weather-app
npm install axios chart.js react-chartjs-2
npm install tailwindcss@latest postcss autoprefixer --save-dev   # Optional but
recommended
npx tailwindcss init -p
```

Step 2: Configure Tailwind
In tailwind.config.js:

```
JavaScript/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./src/**/*.{js,jsx,ts,tsx}"],
  theme: { extend: {} },
  plugins: [],
}
```

In src/index.css:

```
CSS@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step 3: Get Your Free API Key
Go to → https://openweathermap.org/api
Sign up / Login
Get your API Key (APPID) from the dashboard

Step 4: Main Component (src/App.js)
Replace the entire src/App.js with the following code:

```
jsximport React, { useState, useEffect } from 'react';
import axios from 'axios';
import { Line } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  PointElement,
  LineElement,
  Title,
  Tooltip,
  Legend,
} from 'chart.js';

ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip,
Legend);
```

```
const API_KEY = "YOUR_OPENWEATHERMAP_API_KEY_HERE";   // ← Replace with your key

function App() {
  const [city, setCity] = useState("Hyderabad");
  const [currentWeather, setCurrentWeather] = useState(null);
  const [forecast, setForecast] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const fetchWeather = async () => {
    setLoading(true);
    setError("");

    try {
      // Current Weather
      const currentRes = await axios.get(

`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}&units=
metric`
      );

      // 5-day Forecast (used as "historical + future" trend)
      const forecastRes = await axios.get(

`https://api.openweathermap.org/data/2.5/forecast?q=${city}&appid=${API_KEY}&units=
metric`
      );

      setCurrentWeather(currentRes.data);
      setForecast(forecastRes.data);
    } catch (err) {
      setError("City not found or API error. Please try again.");
      console.error(err);
    }
    setLoading(false);
  };

  useEffect(() => {
    fetchWeather();
  }, []);

  // Prepare Chart Data (Daily Temperature Trend)
  const prepareChartData = () => {
    if (!forecast) return { labels: [], datasets: [] };

    const dailyData = {};
    forecast.list.forEach(item => {
      const date = new Date(item.dt * 1000).toLocaleDateString('en-IN', { weekday: 'short',
month: 'short', day: 'numeric' });
      if (!dailyData[date]) {
        dailyData[date] = {
          temp: item.main.temp,
          count: 1
        };
```

```
      } else {
        dailyData[date].temp += item.main.temp;
        dailyData[date].count += 1;
      }
    });

    const labels = Object.keys(dailyData);
    const temps = Object.values(dailyData).map(d => (d.temp / d.count).toFixed(1));

    return {
      labels,
      datasets: [
        {
          label: 'Average Temperature (°C)',
          data: temps,
          borderColor: '#3b82f6',
          backgroundColor: 'rgba(59, 130, 246, 0.2)',
          tension: 0.4,
          fill: true,
        },
      ],
    };
  };

  const chartOptions = {
    responsive: true,
    plugins: {
      legend: { position: 'top' },
      title: { display: true, text: 'Temperature Trend (Last 5 Days + Forecast)' },
    },
    scales: {
      y: { beginAtZero: false, title: { display: true, text: 'Temperature (°C)' } },
    },
  };

  const chartData = prepareChartData();

  return (
    <div className="min-h-screen bg-gradient-to-br from-blue-500 to-indigo-600 text-white
p-6">
      <div className="max-w-4xl mx-auto">
        <h1 className="text-4xl font-bold text-center mb-8">  Weather Dashboard</h1>

        {/* Search Bar */}
        <div className="flex justify-center mb-8">
          <div className="flex w-full max-w-md">
            <input
              type="text"
              value={city}
              onChange={(e) => setCity(e.target.value)}
              placeholder="Enter city name (e.g. Hyderabad)"
              className="flex-1 px-4 py-3 rounded-l-lg text-black focus:outline-none"
            />
            <button
```

```
          onClick={fetchWeather}
          disabled={loading}
          className="bg-white text-blue-600 px-6 py-3 rounded-r-lg font-semibold hover:bg-
gray-100 disabled:opacity-70"
        >
          {loading ? "Fetching..." : "Search"}
        </button>
      </div>
    </div>

    {error && <p className="text-red-300 text-center mb-4">{error}</p>}

    {/* Current Weather */}
    {currentWeather && (
      <div className="bg-white/10 backdrop-blur-lg rounded-2xl p-8 mb-8">
        <div className="flex flex-col md:flex-row items-center justify-between">
          <div>
            <h2 className="text-3xl font-semibold">{currentWeather.name},
{currentWeather.sys.country}</h2>
            <p className="text-6xl font-light mt-4">
              {Math.round(currentWeather.main.temp)}°C
            </p>
            <p className="text-xl capitalize mt-2">
              {currentWeather.weather[0].description}
            </p>
          </div>

          <div className="text-center">
            <img

src={`https://openweathermap.org/img/wn/${currentWeather.weather[0].icon}@4x.png`}
              alt="weather icon"
              className="w-32 h-32"
            />
          </div>

          <div className="grid grid-cols-2 gap-6 text-sm">
            <div>Humidity: <span className="font-
semibold">{currentWeather.main.humidity}%</span></div>
            <div>Wind: <span className="font-semibold">{currentWeather.wind.speed}
m/s</span></div>
            <div>Feels Like: <span className="font-
semibold">{Math.round(currentWeather.main.feels_like)}°C</span></div>
            <div>Pressure: <span className="font-semibold">{currentWeather.main.pressure}
hPa</span></div>
          </div>
        </div>
      </div>
    )}

    {/* Temperature Trend Chart */}
    {forecast && (
      <div className="bg-white/10 backdrop-blur-lg rounded-2xl p-8">
        <h3 className="text-2xl font-semibold mb-6">Temperature Trend</h3>
```

```
          <div className="bg-white rounded-xl p-4">
            <Line data={chartData} options={chartOptions} />
          </div>
          <p className="text-xs text-center mt-4 text-white/70">
            Note: "Historical" view uses recent forecast data (OpenWeather free tier limitation).
            True long-term historical data requires paid subscription.
          </p>
        </div>
      )}
    </div>
  </div>
 );
}

export default App;
```

Replace YOUR_OPENWEATHERMAP_API_KEY_HERE with your actual key.
Run the app:

```
npm start
```

Open → http://localhost:3000

14. Create a TODO application in react with necessary components and deploy it into GitHub.

```
import readline from "readline";

const rl = readline.createInterface({
   input: process.stdin,
   output: process.stdout
});

const todos = [];

const showMenu = () => {
   console.log("\n1: Add a Task");
   console.log("2: View Tasks");
   console.log("3: Exit");
   rl.question("Choose an option: ", handleInput);
};

const handleInput = (option) => {
   option = parseInt(option); // Convert string to number

   if (option === 1) {
      rl.question("Enter the Task: ", (task) => {
         todos.push(task);
         console.log("Task added: ", task);
         showMenu();
      });
   } else if (option === 2) {
      console.log("\n Your Todo Lists");
      if (todos.length === 0) {
         console.log("No tasks yet!");
      } else {
         todos.forEach((task, index) => {
            console.log(`${index + 1}. ${task}`);
         });
      }
      showMenu();
   } else if (option === 3) {
      console.log('Good Bye');
      rl.close();
   } else {
      console.log("Invalid Option. Please try again");
      showMenu();
   }
};

   showMenu();
```

Output:

```
I:\NRDLab\weather\node progs>node ex3.js

1: Add a Task
2: View Tasks
3: Exit
Choose an option: 1
Enter the Task: Attend Meeting at 2PM
Task added:  Attend Meeting at 2PM

1: Add a Task
2: View Tasks
3: Exit
Choose an option: 1
Enter the Task: Extra class scheduled at 3 PM
Task added:  Extra class scheduled at 3 PM

1: Add a Task
2: View Tasks
3: Exit
Choose an option: 2

 Your Todo Lists
1. Attend Meeting at 2PM
2. Extra class scheduled at 3 PM

1: Add a Task
2: View Tasks
3: Exit
Choose an option:
```

5. Viva Voce Questions

1. What is the difference between Flexbox and Grid?
2. What is a responsive web design?
3. How does CSS Grid layout work?
4. What is the use of media queries?
5. What are semantic HTML elements?
6. What is Bootstrap?
7. What is the Bootstrap grid system?
8. What are Bootstrap breakpoints?
9. Difference between container and container-fluid?
10. What are Bootstrap components?
11. What is client-side validation?
12. Difference between client-side and server-side validation?
13. What are form events in JavaScript?
14. What is the use of preventDefault()?
15. How do you validate an email in JavaScript?
16. What are arrow functions in ES6?
17. What is a Promise?
18. Difference between async/await and promises?
19. What is an API?
20. How do you fetch data using fetch()?
21. What is JDBC?
22. What are CRUD operations?
23. Difference between Statement and PreparedStatement?
24. What is a database connection?
25. What is SQL?
26. What is XML?
27. What is DTD?
28. What is XSD?
29. Difference between DTD and XSD?
30. What is XML validation?
31. What is a Servlet?
32. What is MVC architecture?
33. What is a controller in MVC?
34. How does a servlet connect to a database?
35. What is request and response in servlet?
36. What is session tracking?
37. What are cookies?
38. What is HTTP session?
39. Difference between cookies and sessions?
40. What is URL rewriting?
41. What is Node.js?
42. What is the HTTP module?
43. What is the EventEmitter?
44. What does the path module do?
45. What is non-blocking I/O?
46. What is Express.js?
47. What is REST API?
48. What are HTTP methods?
49. What is middleware in Express?
50. What is Postman used for?
51. What is JWT?
52. What are the parts of a JWT?

53. How is JWT used for authentication?
54. Difference between authentication and authorization?
55. What is a secret key?
56. What is React?
57. What are components in React?
58. What is React Router?
59. Difference between functional and class components?
60. What is JSX?
61. What is Chart.js?
62. How do you fetch API data in React?
63. What is useEffect hook?
64. What is state in React?
65. What is data visualization?
66. What is GitHub?
67. How do you deploy a React app?
68. What is version control?
69. What is Git?